



משחק מנקלה מול יריב ממוחשב חכם

שם התלמיד: אלעד ממרוד

ת.ז. הסטודנט: 330925546

שם המנחה: גל בר און

תאריך ההגשה: 23.5.2026

4. מושגים

סעיף זה מגדיר את המושגים הייחודיים הנדרשים להבנת הפרויקט, בהתאם להנחיות הנוהל. אין כאן מושגים כלליים במדעי המחשב, אלא אך ורק מונחים ספציפיים - מונחי משחק המנקלה ומונחי האלגוריתם, בהם נעשה שימוש לאורך ספר זה.

4.1 מונחי משחק המנקלה (Kalah)

- **בור (Pit / גומה):** אחד מ-12 התאים הרגילים בלוח, שישה בצד כל שחקן. בכל בור מונחות אבני משחק; בתחילת המשחק יש בכל בור 4 אבנים.
- **מאגר (Store / קופה):** תא הצבירה של השחקן, אחד בכל קצה לוח. אבן שנכנסה למאגר אינה יוצאת ממנו. מספר האבנים במאגר בסיום המשחק הוא שמקבע את המנצח.
- **זריעה (Sowing):** הפעולה הבסיסית בתור: השחקן מרים את כל האבנים מבור שבחר ומפזר אותן אחת-אחת בתאים העוקבים, נגד כיוון השעון, תוך דילוג על המאגר של היריב.
- **תפיסה / חטיפה (Capture):** אם האבן האחרונה בזריעה נוחתת בבור ריק בצד של השחקן, והבור שמולו (בצד היריב) אינו ריק, השחקן לוקח את האבן שהניח ואת כל אבני הבור הנגדי, ומעביר את כולן למאגרו.
- **תור נוסף (Extra Turn):** אם האבן האחרונה בזריעה נוחתת במאגר של השחקן עצמו, השחקן משחק שוב. רצף של תורות נוספים נקרא **שרשרת (Chain)**.
- **סיום משחק:** כאשר כל ששת הבורות בצד אחד מתרוקנים, המשחק נגמר. כל האבנים שנותרו בצד היריב נאספות אל המאגר של אותו צד.
- **הרעבה (Starvation):** מצב אסטרטגי בו ליריב נותרו מעט מאוד אבנים בבורות, מה שמגביל מאוד את יכולת התמרון שלו.
- **אגירה (Hoarding):** אסטרטגיה של החזקת אבנים בצד שלי במקום לזרוע אותן לעבר צד היריב.

4.2 מונחי האלגוריתם והפתרון

- **מצב לוח (Board State):** תצורה מלאה של הלוח: כמות האבנים בכל אחד מ-14 התאים, ביחד עם זהות השחקן שתורו. בפרויקט זה כל מצב לוח הוא צומת (**Vertex**) במרחב החיפוש.
- **מהלך (Move):** בחירת בור חוקי לזריעה. מהלך הוא קשת (**Edge**) המובילה ממצב לוח אחד למצב לוח אחר.
- **עומק חיפוש:** נמדד ב-plies (**חצי-תור**) - מהלך בודד של שחקן אחד (לדוגמה: "אופק 6" = שישה מהלכים קדימה).
- **שלב משחק (Game Phase):** סיווג של מצב הלוח לאחד מחמישה שלבים (פתיחה, אמצע, סיום, הרעבה, בר-תפיסה), הנקבע על פי מאפייני הלוח ומכתיב אילו משקלים יפעילו פונקציית ההערכה.
- **אוטומט שלבים (FSA - Finite-State Automaton):** רכיב המסווג כל מצב לוח לשלב המשחק שלו באמצעות פונקציית מעבר דטרמיניסטית.
- **היוריסטיקה / פונקציית הערכה (h):** נוסחה המתרגמת מצב לוח למספר יחיד המבטא כמה המצב בטוח עבור הסוכן. ככל ש-h גבוהה יותר - המצב טוב יותר.
- **גורם ההסתעפות (Branching Factor):** מספר המהלכים החוקיים האפשריים ממצב נתון. במנקלה, שישה לכל היותר.
- **פרונטיר (Frontier):** אוסף מצבי הלוח שכבר התגלו אך טרם הורחבו. "הקצה" של מרחב החיפוש.
- **רוחב קרן וקרן (Beam / Beam Width, K):** בכל הסתעפות נשמרים רק K המצבים בעלי ה-h הגבוהה ביותר, והשאר נחתכים. מנגנון זה מצמצם את מרחב החיפוש.
- **אופק עומק (Depth Horizon):** חסם על מספר ה-plies שהחיפוש מעמיק. מצב שהגיע לאופק אינו מורחב אלא מוערך כעלה.
- **טבלת טרנספוזיציות (Transposition Table):** טבלת גיבוב המאחדת מצבי לוח זהים שהושגו דרך רצפי מהלכים שונים, חוסכת הערכות חוזרות, והופכת את מבנה החיפוש מעץ לגרף מכוון ללא-מעגלים (**DAG**).
- **האצלת ערך (Backup):** בסיום החיפוש, ערכו של כל מהלך - שורש, מחושב מהערכים שהתגלו לאורך המסלול שאחריו, ולא רק מהמצב המוערך ביותר שנראה.

5. תיאור הנושא

5.1 תכולת המשחק

המשחק מנוהל על גבי לוח המורכב מ-14 תאים:

- **12 בורות רגילים:** שישה בורות בכל צד, השייכים לשחקן של אותו הצד.
- **2 מאגרים:** תא איסוף אחד בכל קצה של הלוח, השייך לשחקן היושב בצידו.

מצב התחלתי: בכל אחד מ-12 הבורות הרגילים מונחות 4 אבנים (סך הכל 48 אבנים על הלוח). שני המאגרים מתחילים כשהם ריקים.

השחקנים: שחקן 1 הוא שחקן אנושי, ושחקן 2 הוא שחקן ממוחשב (הסוכן החכם). לפני תחילת המשחק, המשתמש קובע במסך הפתיחה מי מבין השחקנים יבצע את המהלך הראשון. עם קביעת המשתמש, התורות מתחלפים ביניהם באופן דינמי על פי חוקי המשחק הבאים:

- **הזריעה (Sowing):** בתורו, השחקן בוחר בור לא ריק מהצד שלו, מרים את כל האבנים מתוכו, ומפזר אותן אחת אחת בתאים העוקבים נגד כיוון השעון. הפיזור מתבצע בבורות שלו, במאגר שלו ובבורות של היריב, תוך דילוג על המאגר של היריב.
- **החטיפה (Capture):** אם האבן האחרונה של פעולת הזריעה נוחתת בבור ריק בצד של השחקן הנוכחי, ובבור של היריב שנמצא בדיוק מולו ישנן אבנים, מתבצעת חטיפה: השחקן לוקח את האבן האחרונה שהניח ואת כל האבנים מהבור הנגדי של היריב, ומעביר את כולן למאגר שלו.
- **תור נוסף (Extra Turn):** אם האבן האחרונה של פעולת הזריעה נוחתת בדיוק במאגר של השחקן עצמו, השחקן זוכה לשחק פעם נוספת. רצף של תורות נוספים יכול להוביל ליצירת שרשרת (Chain) של מהלכים בתוך אותו התור.

5.2 מטרת המשחק

מטרתו המרכזית של כל שחקן היא לצבור כמות מרבית של אבנים בתוך המאגר (קופה) האישי שלו הממוקם בקצה הלוח. מאחר ועל גבי הלוח קיימות 48 אבנים בסך הכל אשר זורמות בצורה חד-כיוונית אל תוך המאגרים ואינן יוצאות מהם, השגת רוב מוחלט של אבנים (25 אבנים או יותר) מבטיחה באופן מתמטי את ניצחונו של השחקן, גם אם המשחק הפיזי עדיין לא הגיע לסיומו הרשמי.

5.3 סיום המשחק

המשחק מגיע לסיומו הפיזי ברגע שבו מתקיים תנאי התרוקנות מלא של אחד הצדדים: כל ששת הבורות הרגילים בצידו של אחד השחקנים ריקים לחלוטין מאבנים. ברגע זה מופעלת שגרת סיום קשיחה (FinalizeBoard): השחקן שבצידו נותרו עדיין אבנים בבורות אוסף את כולן ומעביר אותן ישירות אל המאגר שלו, ובכך מתרוקן הלוח לחלוטין. בתום הפעולה מתבצעת ספירה סופית של שני

המאגרים והכרזה על התוצאה: השחקן בעל מספר האבנים הגבוה ביותר מוכרז כמנצח, ובמצב של שוויון 24 מול 24 המשחק מוכרז כתיקו.

6. רקע תיאורטי

סעיף זה סוקר את העקרונות המדעיים והמודלים הקיימים בעולם לפתרון בעיות מסוג זה, וכיצד הם מתקשרים למשחק המנקלה.

6.1 סיווג המשחק

משחק המנקלה (בגרסת Kalah) שייך למשפחת משחקי הלוח הקומבינטוריים. במונחים של תורת המשחקים, הוא מוגדר על פי המאפיינים הבאים:

- **דו-שחקני:** שני יריבים בלבד משחקים זה מול זה.
- **מידע מלא (Perfect Information):** אין רכיבים נסתרים במשחק. שני השחקנים רואים את מצב הלוח במלואו בכל רגע נתון.
- **דטרמיניסטי:** המשחק אינו כולל אלמנטים של אקראיות, כגון הטלת קוביות או חלוקת קלפים. כל מהלך מוביל למצב לוח יחיד וודאי.
- **סכום-אפס:** הרווח של שחקן אחד הוא בהכרח ההפסד של השחקן השני (ניצחון של האחד הוא הפסד של האחר).

תכונות אלו ממקמות את המנקלה באותה קטגוריה מדעית של משחקים כמו שחמט, דאמה ואיקס-עיגול, ולכן ניתן להחיל עליה את ארגז הכלים המתמטי והחשובי שפותח עבורם.

6.2 ייצוג המשחק כגרף מצבים

הבסיס המתמטי לטיפול במשחק הוא תורת הגרפים. המשחק מיוצג כגרף מכוון שבו:

- **קודקוד (Vertex):** מייצג מצב לוח חוקי (קונפיגורציית האבנים ב-14 התאים ותפקיד השחקן שזהו תורו).
- **קשת (Edge):** מייצגת מהלך חוקי המוביל ממצב לוח אחד למשנהו.

היות ולכל מצב לוח ישנו מצב פתיחה קבוע ויחיד, פריסת כל ההמשכים האפשריים מייצרת עץ משחק. מאחר שהאבנים זורמות באופן חד כיווני אל עבר המאגרים, הגרף הוא מכוון וללא-מעגלים (DAG), תכונה המבטיחה כי כל חיפוש בעץ המשחק יסתיים בתוך מספר סופי של צעדים. במידה והאלגוריתם מזהה שמצבי לוח זהים הושגו דרך רצפי מהלכים שונים, עץ החיפוש "מתקפל" לגרף DAG חסכוני בהרבה.

6.3 גישות קיימות לבחירת מהלך

- **חיפוש ממצה בעצי משחק:** הגישה הקלאסית היא אלגוריתם ה-Minimax, הממדל את היריב כמי שמנסה למקסם את רווחיו (ולמזער את שלנו) וסורק את העץ עד לעלי הסיום. השיפור המקובל עליו הוא "גיזום אפלא-בטא" (Alpha-Beta Pruning), החוסך סריקה של ענפים שאינם משפיעים על השורה התחתונה. שתי השיטות הללו מבטיחות מהלך אופטימלי, אך זמן הריצה שלהן הוא אקספוננציאלי ולכן הן אינן שימושיות במרחבי מצבים

גדולים.

- **פונקציות הערכה היוריסטית (h):** כאשר חיפוש ממצה אינו אפשרי, מעריכים מצבים שאינם סופיים באמצעות היוריסטית - נוסחה מתמטית המחשבת "כמה המצב הנוכחי טוב" בלי להגיע לסוף המשחק. איכות הסוכן תלויה ישירות באיכות פונקציית ההערכה שלו.
- **חיפוש מונחה (Informed Search):** משפחת אלגוריתמים הכוללת את Best-First Search ו-A*, המרחיבה בכל צעד את הצומת המבטיח ביותר על פי ההיוריסטית, במקום לסרוק בצורה עיוורת לפי עומק או רוחב. וריאציה חסכונית בזיכרון של גישה זו היא Beam Search (חיפוש קרן), השומר בכל רמת עומק רק מספר קבוע (K) של מצבים מובילים וחורק את השאר.
- **אוטומטים סופיים (Finite-State Automata):** מודל מתמטי המורכב מאוסף סופי של מצבים ופונקציית מעבר ביניהם. בתחום המשחקים, ניתן להשתמש באוטומט סופי (FSA) כדי לסווג את מצב הלוח הנוכחי לשלב המשחק המתאים לו (כגון פתיחה, אמצע או סיום) ובהתאם לכך לשנות דינמית את אסטרטגיית הנוסחה ההיוריסטית.
- **טבלאות טרנספוזיציה (Transposition Tables):** טכניקת משלים לחיפוש, השומרת בטבלת גיבוב מצבים שכבר חושבו. הדבר מונע הערכה מחדש של מצבים זהים שהושגו בסדר מהלכים שונה ומאחד את עץ החיפוש לגרף DAG.

6.4 מצב הידע על משחק המנקה

בשנת 2000, חוקרים (Irving, Donkers, Uiterwijk) פתרו חישובית את גרסאות Kalah הקטנות. עבור הגרסה הרשמית של המשחק, Kalah(6,4) - שישה בורות ו-4 אבנים, הוכח באמצעות חישוב כבד לאחור (Retrograde Analysis) כי המשחק מוכרע מראש לטובת השחקן הראשון (First-player win). במידה והשחקן הראשון משחק בצורה מושלמת ואופטימלית ללא טעויות, הוא ינצח בהכרח במשחק בתוצאה הסופית. נקודה זו קריטית להגדרת הבעיה האלגוריתמית: ה"פתרון" הוגדר כבסיס נתונים ענקי של סריקה ולא כנוסחה פולינומית פשוטה, ולכן עבור סוכן הפועל בזמן אמת המשחק נותר פתוח ומחייב שילובי חיפוש חכמים.

7. תיאור הבעיה האלגוריתמית

7.1 הגדרת הבעיה

האתגר האלגוריתמי המרכזי בפרויקט הוא לבחור בכל תור של השחקן הממוחשב, את המהלך החוקי הטוב ביותר בזמן אמת (תוך שניות בודדות), ותחת מגבלות הנוהל האוסרות במפורש שימוש באלגוריתמים הקלאסיים העמוקים: Minimax, Alpha-Beta Pruning או Monte Carlo Tree Search (MCTS).

7.2 מדוע זו בעיה של "הערכת מצב"

הנוהל מגדיר בעיה של "הערכת מצב" כבעיה השייכת למחלקת ה-NP, או כבעיה שאין עבורה קריטריון פתרון מתמטי ישיר ופולינומי. בחירת המהלך במנקלה עונה במדויק על הגדרה זו:

- **אין נוסחה ישירה:** לא קיים אלגוריתם מתמטי מהיר המקבל לוח מנקלה כללי ומחזיר מיד את המהלך המושלם. גם העובדה שהמשחק "פתור" (סעיף 6.4) אינה מספקת נוסחה כזו, שכן הפתרון נשען על פריסה מלאה מראש של מיליארדי מצבים.
- **מרחב מצבים מתפוצץ (Explosive State Space):** מכיוון שמכל עמדה נתונה קיימים לכל היותר שישה מהלכים חוקיים (גורם ההסתעפות של עץ המשחק הוא b קטן או שווה ל-6). כתוצאה מכך, חיפוש המעמיק d מהלכים קדימה נוגע בקירוב ב- d^6 מצבים פוטנציאליים (גורם ההסתעפות בחזקת העומק). כבר בעומק של שישה מהלכים (אופק של 6 plies) מדובר ב- $6^6 = 46,656$ מצבים שונים, והמספר ממשיך לגדול בקצב אקספוננציאלי מהיר ככל שהעומק גדל. מסיבה זו, ביצוע חיפוש מלא וממצה אינו ישים במסגרת זמן ריצה סביר במחשב אישי, והבעיה מוגדרת במפורש כבעיית "הערכת מצב" הדורשת פיתוח נוסחת דירוג היוריסטית ולא פתרון מתמטי ישיר.

7.3 מורכבויות ייחודיות למשחק המנקלה

מעבר לגודל המרחב, חוקי המשחק מייצרים שני אתגרים חישוביים ייחודיים:

- **שרשור תורות נוספים (Chaining):** מהלך שמסתיים במאגר מעניק תור נוסף, מה שמאפשר לשחקן לבצע סדרה ארוכה של מהלכים רצופים במסגרת אותו ה"תור". עבור האלגוריתם, "צעד אחד קדימה" אינו מהלך בודד אלא שרשרת מהלכים שלמה שיש לחשב את סופה, לפני שהתור עובר ליריב.
- **מלכודות תפיסה (Traps):** מהלך יכול להיראות רווחי מאוד בטווח המיידי (למשל, צבירת אבנים קלה), אך הוא עלול להשאיר בורות ריקים בצד של הסוכן שיאפשרו ליריב לבצע מהלך חטיפה הרסני מיד לאחר מכן. לכן, הערכת המצב חייבת לקחת בחשבון את תגובת היריב באופן מובהק.

7.4 המסקנה האלגוריתמית

מאחר שחיפוש מלא אינו ישים, והאלגוריתמים הממצים הרגילים (Minimax/AB) אסורים על פי הנוהל (סעיף 13) ועל פי הצעת הפרויקט המקורית, הפתרון מחייב שילוב של:

1. **חיפוש מקומי מוגבל (Look-ahead):** הגעה עד אופק עומק מוגדר בזמן קצר.
2. **פונקציית הערכה היוריסטית משוכללת:** להערכת מצב שאינם סופיים על פי שלבי המשחק השונים (באמצעות מודל אוטומט שלבים).
3. **צמצום רוחב החיפוש:** שימוש באלגוריתם חיפוש מידע מוכוון דוגמת Beam Search כדי להתגבר על הפיצוץ האקספוננציאלי.

פירוט הפתרון שנבחר מופיע בסעיפים 8 ו-9.

8. סקירת אלגוריתמים בתחום הבעיה

סעיף זה סוקר את משפחות האלגוריתמים הרלוונטיות לבחירת מהלך במשחקי לוח, ובוחן את מידת התאמתן לבעיית הפרויקט הנוכחית, מתוך מטרה להסביר אילו מנגנונים נפסלו ומדוע, ואיזה כיוון אלגוריתמי נבחר לפתרון.

8.1 אלגוריתמים של חיפוש ממצה (אסורים)

- **Minimax (מינימקס):** האלגוריתם הקלאסי לפתרון משחקי לוח דו-שחקניים. הוא סורק את עץ המשחק עד לעומק קבוע מראש: במצבים של הסוכן הוא בוחר את המהלך בעל הערך המרבי (MAX), ובמצבי היריב הוא מניח בחירה בערך המזערי (MIN), ומעלה את הערכים הללו מהעלים חזרה אל השורש.

- **יתרון:** מבטיח מהלך אופטימלי תיאורטית מול יריב שמשחק בצורה מושלמת.
- **חסרון:** זמן ריצה אקספוננציאלי - $O(b^d)$. במשחק המנקה, חוק התור הנוסף מאיץ דרסטית את קצב ההסתעפות ויוצר התפוצצות קומבינטורית, מה שמונע סריקה לעומק משמעותי במסגרת זמן אמת.

- **Alpha-Beta Pruning (גיזום אלפא-בטא):** שיפור מובנה לאלגוריתם המינימקס, אשר מזהה וגוזם ענפים בעץ החיפוש שאין להם סיכוי להשפיע על ההחלטה הסופית של השורש.
- **יתרון:** מקצר את מרחב החיפוש בצורה משמעותית.
- **חסרון:** במקרה הגרוע זמן הריצה נותר אקספוננציאלי. יעילות הגיזום תלויה לחלוטין בסדר בדיקת המהלכים (Move Ordering), וללא מנגנון מיון מקדים כבד, האלגוריתם יסרוק כמות עצומה של מצבים מיותרים.

- **MCTS (Monte Carlo Tree Search - חיפוש עץ מונטה-קרלו):** אלגוריתם המרחיב את עץ המשחק בהדרגה על בסיס סימולציות אקראיות של משחקים (Playouts) עד להגעה למצב סיום, תוך העדפת כיוונים שהניבו סטטיסטיק תוצאות טובות יותר.
- **יתרון:** האלגוריתם אינו דורש פיתוח מורכב של נוסחה היוריסטית ייעודית ומותאמת אישית ללוח, מאחר שהוא מגלה את איכות המצבים בכוחות עצמו דרך הרצת משחקים סטטיסטיים עצמאיים. הדבר הופך אותו ליעיל ביותר עבור משחקים בעלי חוקים פשוטים אך מרחב מצבים עמוק ומורכב.
- **חסרון:** האלגוריתם נשען על סימולציות אקראיות, ולכן מפספס מלכודות חטיפה חדות במשחק טקטי ונוקשה כמו מנקה. בנוסף, הסתמכות על אקראיות סותרת ישירות את הנחיות הנוהל, הדורשות הערכת מצב דטרמיניסטית ומבוססת מידע בלבד.

הערה מתוך הצעת הפרויקט: שלושת האלגוריתמים הללו נפסלו מראש גם במסגרת הצעת הפרויקט המאושרת, שניתחה את אלגוריתם המינימקס בניתוח ה-SWOT שלה וסימנה אותו כחיסרון מובנה שיש להימנע ממנו.

8.2 אלגוריתם חמדן נאיבי (Naive Greedy)

אלגוריתם זה אינו מבצע כל הסתכלות קדימה אל עבר העתיד (Look-ahead). הוא בוחן אך ורק את המצב הנוכחי שעל הלוח, ומבצע באופן מיידי את המהלך שמעניק לו את מספר האבנים הגבוה ביותר באותו הרגע (למשל, העדפת מהלך שמבצע הפיצה מיידי או תור נוסף יחיד).

- **יתרון:** פשוט מאוד למימוש ופועל במהירות חישובית כמעט אפסית.

- **חיסרון:** סובל מ"קוצר ראייה" חריף. האלגוריתם אינו מסוגל לחזות את תגובות הנגד של היריב, נופל בקלות למלכודות הפיצה שטומנים לו, ומציג רמת משחק חלשה ביותר. האלגוריתם אמנם אינו אסור על פי הנוהל, אך הוא אינו עומד ברמה האלגוריתמית הנדרשת מפרויקט הגמר.

8.3 חיפוש מוכוון מידע (Informed Search)

משפחת אלגוריתמים המשתמשת בפונקציית הערכה היוריסטית כדי לכוון את מאמצי החיפוש לעבר הענפים המבטיחים ביותר. גישה זו אינה נשענת על אקראיות, ואינה נמנית עם האלגוריתמים שנאסרו בנוהל:

- **Best-First Search:** מנגנון המחזיק את כל מצבי הלוח שהתגלו בתוך תור יחיד (Frontier), ומבצע בכל שלב הרחבה של הצומת בעל הערך ההיוריסטי הגבוה ביותר, במקום לסרוק באופן עיוור לפי סדר עומק או רוחב.
- **A*:** וריאציה של חיפוש Best-First, המדרגת את מצבי הלוח על פי שילוב של העלות שהושקעה עד כה להגעה לצומת (g) ביחד עם ההערכה ההיוריסטית של המרחק שנותר מהצומת אל היעד (h).
- **Beam Search (חיפוש קרן):** וריאציה חסכונית במשאבי זיכרון של חיפוש מוכוון מידע. האלגוריתם מגדיר רוחב קרן קבוע (K), ובכל רמת עומק של פריסת המהלכים הוא שומר אך ורק את K המצבים המובילים שקיבלו את הציון ההיוריסטי הגבוה ביותר, בעוד ששאר המצבים נחתכים ונזרקים. מנגנון זה מתאים במיוחד להתמודדות עם מרחבי מצבים אקספוננציאליים תחת מגבלת זמן ומשאבים קשיחה.

8.4 טבלת השוואה אלגוריתמית

מתאים לדרישות הפרויקט?	זמן ריצה	עיקרון פעולה	אלגוריתם
לא - אסור במפורש על פי הנוהל.	אקספוננציאלי	חיפוש ממצה, בניית שכבות MAX/MIN	Minimax
לא - אסור במפורש על פי הנוהל.	אקספוננציאלי	מינימקס בשילוב גיזום ענפים לא רלוונטיים	Alpha-Beta

לא - אסור בנוהל ומבוסס על אקראיות.	תלוי בתקציב הזמן	סימולציות אקראיות והערכה סטטיסטית	MCTS
לא - משחק חלש מדי ("קוצר ראייה").	מהיר מאוד	מקסום הרווח המייד, ללא מבט קדימה	חמדן נאיבי
כן - מהווה בסיס חוקי לפתרון.	מבוקר על ידי תקציב המצבים	הרחבת המצב המבטיח ביותר לפי היוריסטיקה	Best-First / *A
כן - בסיס מצוין לפתרון תחת מגבלת משאבים.	מבוקר על ידי תקציב ורוחב הקרן	חיפוש מכוון מידע בשילוב חיתוך ל-K מצבים	Beam Search

8.5 מסקנה

שלושת אלגוריתמי החיפוש הממצה והסטטיסטי (Minimax, Alpha-Beta, MCTS) נפסלים לחלוטין - הן מטעמי עמידה בדרישות החובה של הנוהל והן בשל סיבוכיות זמן הריצה והסתמכות על אקראיות. אלגוריתם החמדן הנאיבי אמנם חוקי, אך פשטני וחלש מכדי להציב אתגר אמיתי.

לפיכך, הכיוון האלגוריתמי שנבחר בפרויקט זה נשען על משפחת החיפוש מכוון המידע, ומבוסס על אלגוריתם **Best-First Beam Search** המשולב עם מודל חיזוי לתגובת היריב, פונקציית הערכה היוריסטית דינמית ואוטומט שלבים (FSA). תיאור מפורט ומלא של האסטרטגיה הנבחרת מופיע בסעיף 9.

9. אסטרטגיית הפתרון הנבחרת

9.1 סקירה כללית

האסטרטגיה הנבחרת לפרויקט זה היא אלגוריתם חיפוש וסריקה מתקדמת מסוג **חיפוש קרן מיטבי מודע-שלבים (Phase-Aware Best-First Beam Search)**. שיטה זו משתייכת למשפחת האלגוריתמים מכווני-המידע (Informed Search) לניתוח ובחירת מהלכים על גבי גרף מצבים, כפי שנסקרה בסעיף 8. המערכת מייצגת את המשחק כאוטומט של גרף-מצבים מכוון וללא מעגלים (DAG), שבו כל צומת הוא קונפיגורציית לוח חוקית, וכל קשת היא מהלך זריעה חוקי. הצעת הפרויקט המאושרת בחרה באלגוריתם "Adversarial Greedy Search", וכן נתנה ברשימת טכניקות ה-AI שלה את "Greedy Best-First Search". האסטרטגיה המוצגת כאן,

"Phase-Aware Best-First Beam Search", מאחדת את שתי הטכניקות שאושרו: מודל היריב החמדן ממומש במדויק במתודת SimulateGreedyOpponentMove ומשמש כשכבת חיזוי תגובת היריב, וסביבו נבנתה מעטפת חיפוש Best-First מכוונת-מידע. בנוסף, מנגנון ההבחנה בין רמות הקושי שונה מהמתואר בהצעה (רעש אקראי) למנגנון דטרמיניסטי מבוסס עומק חיפוש – שינוי המתחייב מדרישת הדטרמיניזם של הנוהל. אין כאן שינוי תכולה או חריגה מגבולות הפרויקט שאושרו, אלא הבשלה והרחבה של הרכיבים שאושרו.

כדי להתגבר על הפיצוץ האקספוננציאלי של מרחב המצבים מבלי להפר את הגבולות הנוהל, המנוע פועל באמצעות ארכיטקטורה מובנית של ארבע שכבות עבודה משולבות:

1. **אוטומט שלבי המשחק (FSA):** מסווג את עמדת הלוח לאחד מחמישה שלבים אסטרטגיים שונים (סעיף 9.3).
2. **פונקציית הערכה היוריסטית דינמית:** מתרגמת את מצב הלוח לציון מספרי משוקלל, כאשר המשקולות משתנות דינמית בהתאם לשלב שזיהה האוטומט (סעיף 9.4).
3. **מנוע חיפוש Best-First Beam Search:** מנהל את חזית החיפוש ומבצע חיתוך רחב (Pruning) אגרסיבי כדי לשמור על יעילות משאבים (סעיף 9.5).
4. **טבלת טרנספוזיציות (Transposition Table):** מאחדת מסלולי חישוב זהים ומונעת הערכות היוריסטיות כפולות (סעיף 9.6).

עבור דרגת הקושי הגבוהה ביותר (Hard), המערכת מוסיפה על גבי ארבע השכבות הללו שני מנגנונים ייחודיים ובלעדיים: **רכיב שומר-הגנה (Capture Guard)** למניעת הפקרת אבנים (סעיף 9.8), ו**פותר סיום מדויק (Endgame Solver)** לחישוב מתמטי מוחלט של מהלכי ההכרעה (סעיף 9.9).

תהליך קבלת ההחלטות הכללי של הסוכן בתורו מתבצע בסדר הבא:

- במידה ודרגת הקושי היא Hard, והלוח נכנס לשלב סיום (פחות מ-12 אבנים בבורות), מופעל מיידיית פותר הסיום הממצה. אם נמצא פתרון מתמטי, המהלך המנצח מוחזר ישירות והתהליך מסתיים.
- בכל תרחיש אחר, מופעל מנוע החיפוש המקורב. המנוע מקצה תקציב מצבים שווה לכל אחד ממהלכי השורש החוקיים, מריץ את חישוב הקרן ומפעיל מעבר האצלת ערכים לאחור (Backup) כדי לקבוע את הציון של כל מסלול.
- במידה ודרגת הקושי היא Hard, המהלך הנבחר עובר סינון נוסף דרך רכיב ה-Capture Guard. אם מתגלה שהמהלך חושף את הסוכן לחטיפה הרסנית מצד היריב, המנגנון מבצע עקיפה (Override) ומחזיר מהלך אלטרנטיבי המגן על הלוח.

הפסאודו-קוד הפורמלי והמלא של זרימת האלגוריתם מתועד בסעיף 14.

9.2 מודל היריב: חמדן, לא מושלם

לב האסטרטגיה מבוסס על האופן שבו הסוכן ממדל את התנהגות היריב. אלגוריתם המינימקס הקלאסי (סעיף 8.1) מניח "יריב מושלם" (Perfect Opponent), כלומר שחקן בעל משאבי חישוב אינסופיים המחשב את העץ עד לסוף המשחק ובוחר תמיד במהלך שממקסם את הפסדנו. גישה זו מנותקת מהמציאות של משחק מול שחקן אנושי ומנוגדת להנחיות.

פרויקט זה נוקט בגישה שונה וממדל את היריב כשחקן חמדן מקומי (Adversarial Greedy). המודל מניח כי השחקן שמולנו פועל מתוך אנוכיות וחמדנות קצרת-רואי: הוא אינו סורק ענפי חישוב עמוקים, אלא יבחר בכל תור במהלך המעניק לו את הרווח הגבוה והמיידית ביותר על גבי הלוח באותו הרגע (כגון העדפת מהלך המבצע חטיפה או כזה המעניק תור נוסף).

בחירה זו מוגנת ומנומקת בשני מישורים:

- **ריאליזם הנדסי:** שחקן אנושי ממוצע אינו מסוגל לחשב בראשו מיליוני מצבים קדימה בזמן אמת. מודל יריב חמדן פשוט מייצג בצורה מדויקת בהרבה את תגובות הנגד של המשתמש, ובכך מאפשר לסוכן לחזות מלכודות אנושיות בצורה יעילה.

- **תאימות מלאה להנחיות הנוהל:** הסרת שכבת המינימזציה (שכבת ה-min בעץ החיפוש) מוציאה את האלגוריתם לחלוטין מגדר המינימקס האסור (סעיף 13 בנוהל), שכן עץ החיפוש אינו מחליף שכבות של max ו-min אלא מבוסס על פריסה חד-צדדית המשולבת עם חיזוי מסלול יחיד של תגובה חמדנית.

מודל זה אינו רכיב חדש, אלא מהווה הבשלה ומימוש מלא של אלגוריתם ה-"Adversarial Greedy Search" שהובטח ואושר במסמך הצעת הפרויקט המקורית. המודל מוטמע כמתודה פנימית מוגנת בקוד (MancalaProject.Core/BeamSearch.cs) תחת הפונקציה (SimulateGreedyOpponentMove). בכל פעם שסריקת המנוע מגיעה לצומת המייצג את תורו של היריב, המערכת אינה מפצלת את העץ לכל הכיוונים, אלא מפעילה את מודל היריב ומסמלצת מהלך יחיד נבחר המייצג את תגובתו הריאליסטית, וממנו ממשיכה את החיפוש קדימה. האסטרטגיה הנוכחית אינה מחליפה את הצעת המקור, אלא בונה מעטפת חיפוש חכמה סביבה.

9.3 שכבה 1: אוטומט שלבי המשחק (FSA)

אוטומט מצבים סופי (Finite-State Automaton) מבוסס על הטיפוס enum GamePhase ומשמש כשכבה הראשונה של המנוע. האוטומט מנתח את מאפייני פיזור האבנים על גבי הלוח ומסווג כל עמדה לאחד מחמישה שלבים אסטרטגיים קבועים:

שלב (State)	תנאי כניסה חישובי	משמעות אסטרטגית ופעולה
Opening	סך האבנים ב-12 הבורות 36 ומעלה	שלב פתיחת המשחק. המגרש עמוס באבנים, והדגש מושם על תפיסת בורות מפתח וייצול פוטנציאל חטיפות עתידי.
Midgame	סך האבנים ב-12 הבורות בין 12 ל-36	שלב אמצע המשחק. ניהול איזון עדין בין צבירת נקודות במאגר, שמירה על שליטה בבורות, ומניעת איומי יריב.

שלב הסיום הפיזי. האבנים על הלוח מועטות, המשחק הופך טקטי והמשקל עובר להכרעה מוחלטת של הניקוד הממומש.	סך האבנים ב-12 הבורות קטן מ-12	Endgame
שלב הרעבה. היריב חנוק ובקושי מחזיק באבנים לתמרון. המטרה היא למנוע הזנת אבנים לצד שלו כדי לאלץ אותו לסיים את המשחק.	סך האבנים בבורות היריב 3 ומטה	StarvationMode
שלב שרשרת חטיפות. עמדת הלוח רגישה ומכילה הזדמנויות חטיפה מרובות זמינות. המשקל עובר למיקסום ותעדוף חטיפות.	קיום 2 או יותר איומי חטיפה פעילים	CaptureChain

האוטומט הוא חסר-זיכרון (Stateless) ומחשב את מצבו מחדש בכל פנייה בזמן קבוע של $O(1)$ על ידי סריקה חסומה של 14 התאים, ללא שמירת מצב בין מהלכים. במידה ועמדת לוח מסוימת מקיימת את תנאי הכניסה של יותר משלב אחד במקביל, האוטומט מכריע את הסיווג על פי סדר קדימויות קשיח המייצג את הדחיפות האסטרטגית של העמדה:

StarvationMode ← CaptureChain ← Endgame ← Midgame ← Opening

סיווג דינמי זה הופך את פונקציית ההערכה ההיוריסטית לפונקציה חלקית (Piecewise Function) המותאמת במדויק לקצב ומצב המשחק, במקום להישען על נוסחה סטטית אחת קשיחה לכל אורך הדרך.

9.4 שכבה 2: פונקציית ההערכה ההיוריסטית הדינמית

נוסחת ההערכה ההיוריסטית (h) מתרגמת את הצומת הנסרק למספר ממשי יחיד. ככל שהציון גבוה יותר, המצב נחשב בטוח ומבטיח יותר עבור הסוכן הממוחשב. הנוסחה מוגדרת כך:

$$h(\text{state}) = w1 * \Delta S + w2 * (\text{myCapture} - \text{oppCapture}) + w3 * D + \text{ChainBonus} - \text{StarvationPenalty}$$

מרכיבי ליבת הנוסחה:

- **ΔS (הפרש הניקוד הממומש):** מספר האבנים במאגר הסוכן פחות מספר האבנים במאגר היריב. זהו המדד המרכזי לניצחון.

- **$(\text{myCapture} - \text{oppCapture})$ (פוטנציאל חטיפות נטו):** חישוב אסימטרי משוקלל של איומי החטיפה הקיימים על הלוח. המערכת מעניקה משקל גבוה (ציון 2.0) לאיומי חטיפה אקטיביים הניתנים למימוש מיידי בתור הנוכחי, ומשקל נמוך (ציון 0.5) לאיומים פסיביים

הדורשים מהלך הכנה מוקדם. איומי חטיפה של היריב מבוצעים בסימן שלילי כדי לקזז את הציון.

- **D (שליטה בלוח):** סך כל האבנים המונחות בבורות הרגילים של הסוכן. משמש כ"תחמושת" חיונית ליכולת תמרון וניהול מהלכים עתידיים.

דרישת "המשקולות הדינמיות" מיושמת על ידי כך שכל שלב משחק שהוגדר ב-FSA טוען וקטור משקולות (w_1, w_2, w_3) שונה לחלוטין, המשנה את אופי ההתנהגות של הסוכן:

רכיבים מיוחדים משולבים	משקולת w_3 (שליטה)	משקולת w_2 (חטיפה)	משקולת w_1 (ניקוד)	שלב משחק פעיל
—	0.4	0.7	1.0	Opening
—	0.3	0.6	1.0	Midgame
משקל הניקוד עולה ל-1.5 (כל אבן במאגר קריטית). משקל השליטה צונח ל-0.1.	0.1	0.4	1.5	Endgame
משקל השליטה הופך לשלילי (-0.2); אגירת אבנים בצד שלי מאריכה את חיי היריב ונענשת.	-0.2	0.6	1.0	StarvationMode
משקל החטיפה עולה ל-1.2 ומופעל רכיב ה-ChainBonus.	0.3	1.2	1.0	CaptureChain

רכיבים מיוחדים מותנים:

- **ChainBonus (בונוס שרשרת):** מופעל בלעדית בשלב ה-CaptureChain ומעניק תוספת של 1.5 נקודות לכל מהלך המייצר רצף של איומי חטיפה עוקבים, כדי לעודד את הסוכן לגרור את המשחק למערכה אגרסיבית.
- **StarvationPenalty (קנס הרעבה):** מופעל (בכל השלבים) כאשר ליריב נותרות 3 אבנים או פחות בבורות שלו. הרכיב מטיל קנס קבוע של 3.0 נקודות עבור כל אבן של היריב, כדי

למנוע מהסוכן לבצע מהלכים המזינים אבנים לצד של היריב ובכך "מצילים" אותו מהחנק האסטרטגי.

קיצור דרך טרמינלי: כאשר מנוע ההערכה מזהה מצב סיום משחק רשמי ($IsGameOver == true$), הנוסחה ההיוריסטית מנוטרלת לחלוטין. המתודה מפעילה את פונקציית הסיום, מחשבת את הפרש האבנים האמיתי והסופי בין המאגרים, ומכפילה אותו בגורם קשיח ענקי של 10,000. צעד זה מבטיח כי עמדות המובילות לניצחון מוחלט יועדפו תמיד על פני כל עמדה היוריסטית משוערת אחרת, ומצבי הפסד יידחו לחלוטין.

9.5 שכבה 3: מנוע החיפוש Best-First Beam Search

מנוע החיפוש אחראי על פריסת המהלכים וסריקת מרחב המצבים בצורה מבוקרת. המנוע מבוסס על העקרונות המבניים הבאים:

- **ניהול חזית החיפוש (Frontier):** מנוהל באמצעות מבנה נתונים עצמי מסוג ערימת מינימום (MinHeap). הצמתים מוכנסים לערימה עם עדיפות שלילית ($h-$), תכונה המבטיחה כי הצומת בעל הציון ההיוריסטי הגבוה ביותר צף תמיד לראש המערך ונשלף ראשון ($O(\log n)$), ובכך מייצר התנהגות של חיפוש מוכוון מידע (Best-First).

- **חלוקת תקציב שוויונית בין מהלכי השורש:** המנוע קולט את תקציב הצמתים הכולל המוקצה לדרגת הקושי ומחלק אותו בצורה שווה לחלוטין בין כל מהלכי השורש החוקיים הזמינים לסוכן באותו התור. חלוקה זו מיושמת באופן מפורש בקוד באמצעות הקצאת לולאה ייעודית, ומבטיחה כי כל תת-עץ של מהלך פתיחה ייחקר לאותו עומק ובאותה רמת השקעה משאבית, דבר המאפשר לבצע השוואה הוגנת ומדויקת ביניהם בשלב ה-Backup ללא עיוותים הנגרמים מחקירה לא שוויונית.

- **חיתוך רוחב (Beam Pruning – פריסה אסימטרית):** זרימת החיפוש מפרידה בין צעדי הסוכן לצעדי היריב:
 - בתורו של הסוכן: המנוע מייצר את כל המהלכים החוקיים, מדרג אותם ומותיר בחזית החיפוש אך ורק את K הצמתים המובילים בעלי הציון ההיוריסטי הגבוה ביותר (רוחב הקרן), בעוד ששאר הענפים נחתכים ונזרקים מיידית.
 - בתורו של היריב: המנוע אינו מפצל את העץ לכל הכיוונים, אלא מפעיל את מודל היריב החמדן ומייצר צומת המשך יחיד (Single-child path) המייצר את מסלול החיזוי הישיר של תגובתו הצפויה.

- **כיווץ שרשרת תורות נוספים:** חוק "תור נוסף" במנקלה מאפשר לבצע שרשרת מהלכים באותו התור. המנוע מזהה מצבים אלו ומריץ לולאת פיזור פנימית המכווצת את כל רצף המהלכים הללו לכדי קשת אחת ארוכה ומאוחדת בגרף המצבים. עבור האוטומט, צעד אחד קדימה מייצג תמיד "תור שלם" שהסתיים והועבר לצד השני, ולא מהלך פיזור בודד.

- **אופק עומק קשיח:** החיפוש מוגדר עם חסם עומק מקסימלי (על פי דרגת הקושי הפעילה). צומת שהגיע לאופק העומק אינו מורחב יותר לחזית, אלא מוגדר כעלה ומקבל את הציון ההיוריסטי המידי שלו, שבו נשמר מודל היריב החמדן בצורה המהימנה ביותר.

החיפוש מסתיים בצורה דטרמיניסטית ברגע שתקציב הצמתים של תת-העץ נוצל במלואו או כאשר חזית החיפוש התרוקנה.

9.6 שכבה 4: טבלת הטרנספוזיציות

כדי למנוע בזבוז משאבי עיבוד יקרים על הערכות היוריסטיקות חוזרות של מצבי לוח זהים שהושגו בסדר מהלכים שונה במהלך החיפוש המקורב, המנוע מפעיל טבלת גיבוב מובנית מסוג `<Dictionary>long, double`.

הערך המשמש כמפתח מחושב באמצעות אלגוריתם הגיבוב המהיר **(FNV-1a (64-bit)** המריץ פונקציית כפל ו-XOR מתמטית רציפה על גבי מערך 14 התאים של הלוח, בשילוב זהות השחקן התורן. ערך זה מיוצר בזמן קבוע של $O(1)$. פונקציית הגיבוב המתמטית משיגה בדיוק את אותו פיזור ואותו יעד הנדסי שהובטח בהצעת הפרויקט תחת השם Zobrist Hashing (איחוד מצבי לוח זהים בגרף ומניעת כפילויות), אך היא חסכונית ומבוצרת בהרבה, שכן היא אינה דורשת הקצאה והחזקה קבועה בזיכרון של מערכי מספרים אקראיים ענקיים כפי שמאפיינת את טכניקת ה-Zobrist.

הטבלה "מקפלת" את מבנה עץ החיפוש הרגיל והופכת אותו לגרף מכוון ללא-מעגלים (**DAG**), המבטיח שכל עמדה ייחודית תחושב פעם אחת בלבד.

9.7 מהלך בחירת המהלך: שלב ה-Backup

בסיום פעולת החיפוש ופריסת הצמתים, מופעל מעבר האצלת ערכים לאחור (**Backup**) על גבי עץ החיפוש שנסרק. המעבר מבוצע באמצעות מתודה רקורסיבית הנעה מהעלים חלפי מעלה אל עבר צמתי השורש, ומחשבת לכל צומת את ערכו הריאליסטי בהתאם לכללי מודל המשחק:

- **בצומת עלה (צומת שלא הורחב או שהגיע לאופק):** הערך המואצל הוא הציון ההיוריסטי הישיר של המצב ($Value = h$).
- **בצומת השייך לסוכן (שכבת MAX מקומית):** הערך המואצל הוא הערך המקסימלי מבין כל ערכי ה-Backup של צאצאיו שנסרקו בקרן ($Value = \max(Children)$). הסוכן מניח כי בתורו הוא יבחר במסלול המבטיח לו את הציון הגבוה ביותר.
- **בצומת השייך ליריב (שכבת חיזוי):** הצומת מכיל צאצא יחיד המייצר את מסלול התגובה החמדנית שנחזתה במודל. הערך המואצל של הצומת שווה במדויק לערך ה-Backup של אותו ילד יחיד, ללא ביצוע פעולת מינימיזציה.

המהלך הנבחר שיוחזר ויבוצע על הלוח הראשי הוא מהלך השורש שקיבל את ערך ה-Backup הגבוה ביותר בתום המעבר. מנגנון זה מחליף את הבחירה הנאיבית של "הצומת הטוב ביותר שנראה אי פעם בעץ", ומבטיח כי המהלך שנבחר מוביל למסלול יציב ומאזן לאורך זמן, תוך דחיית מסלולים המכילים "פנטזיות" רגעיות שביניהם ממוקמים מהלכי כשל הרסניים.

9.8 מנגנון שומר-ההגנה (Capture Guard – בלעדי ל-Hard)

מנוע החיפוש המקורב חסום בעומק וברוחב הקרן, ועלול במצבים מסוימים ליפול ל"אפקט האופק" - מצב שבו מהלך נראה רווחי בטווח המיידי של העץ, אך הוא מפקיר בור ריק המאפשר ליריב לבצע חטיפה הרסנית מיד בצעד הבא שאחרי האופק הנסרק.

על מנת לספק רשת ביטחון מוחלטת, רמת הקושי **Hard** מפעילה שכבת סינון קשיחה עליונה (`ApplyCaptureGuard`) מיד לאחר בחירת המהלך באלגוריתם:

- **בדיקת חשיפה:** המנגנון מריץ סימולציה מהירה של המהלך שנבחר ובודק את תגובת הנגד המיידית של היריב האנושי בתור הבא.

- **תנאי הפסילה:** אם מתגלה כי המהלך שנבחר משאיר ליריב הזדמנות לבצע חטיפה משמעותית (חטיפה של 3 אבנים או יותר בחינם), והסוכן עצמו אינו מבצע חטיפה נגדית גדולה באותו המהלך - המהלך מוגדר כ"מהלך כשל" ונפסל מיידית.

- **עקיפה והגנה (Override):** המנגנון סורק את שאר מהלכי השורש החוקיים, מוצא את המהלך האלטרנטיבי המעניק את רמת החשיפה המינימלית ביותר ליריב (המהלך המגן על הלוח), ומחזיר אותו לביצוע במקום המהלך המקורי שנפסל.

רכיב ה-Capture Guard מהווה חוק מתמטי קשיח המגן על הסוכן מפני פנטזיות אסטרטגיות ומבטל לחלוטין את מהלכי הפקרת הבורות שהתגלו ועלו במהלך סבבי בדיקות האיכות הידניים של רמת ה-Hard, ובכך הופך אותה ליריב חסין, עקבי ומאתגר.

9.9 פותר מצבי הסיום (Endgame Solver – בלעדי ל-Hard)

השכבה החמישית והעליונה של המערכת, הפועלת בלעדית ברמת קושי Hard, היא פותר הסיום הממצה (EndgameSolver). כאשר כמות האבנים הכוללת ב-12 הבורות הרגילים יורדת ל-12 ומטה אבנים, מרחב המצבים של המשחק מצטמצם בצורה משמעותית והמשחק הופך טקטי לחלוטין. ברגע זה, המנוע מפעיל את פותר הסיום ומדלג על חישוב ה-Beam Search המקורב:

- **סריקה ממצה וערכים מוחלטים:** הפותר מריץ סריקה רקורסיבית ממצה המגיעה עד לעלי הסיום המוחלטים של המשחק (`IsGameOver == true`). ברמת העלים מופעלת פעולת ה-FinalizeBoard ומחושב הפרש הניקוד הסופי האמיתי שיושג בלוח. ערך זה אינו הערכה היוריסטית משוערת אלא מספר מתמטי מוחלט וודאי.

- **ארכיטקטורה חד-צדדית דטרמיניסטית:** הפותר שומר על חוקי המודל ומבצע בחירת מקסימום (max) על מהלכי הסוכן, בשילוב חיזוי מהלך חמדני יחיד של היריב (`SimulateGreedyOpponentMove`), ללא פריסת שכבת min מלאה. תכונה זו שומרת עליו מחוץ לגדר אלגוריתם המינימקס האסור בנוהל.

- **טבלת זיכרון חסינת התנגשויות (Memoization):** הפותר מנהל טבלת זיכרון ייעודית מסוג `Dictionary<string, int>` המבוססת על מחרוזת מלאה של 14 התאים והתור הנוכחי (ראה סעיף 12.7). הטבלה מונעת חישובים כפולים ומאחדת את עץ הסיום לגרף DAG מוחלט.

- **תקציב בטיחות:** כדי להבטיח שהתוכנה לעולם לא תיכנס ללולאה אינסופית או תגרום לתקיעת ממשק המשתמש, נקבע חסם קשיח (Safety Budget) של עד 250,000 מצבי לוח ייחודיים בסריקה. במידה וגודל העץ חורג מהתקציב, הפונקציה עוצרת, מבצעת נסיגה (Fallback) חנינית ואוטומטית אל חיפוש ה-Beam הרגיל.

שילוב פותר הסיום מעניק לרמת ה-Hard יתרון עוצמתי מוחלט - היא פשוט אינה מסוגלת לבצע טעויות חישוב בשללב סיום המשחק ומנווטת את האבנים להכרעה מתמטית מושלמת.

9.10 פרמטרי שלוש רמות הקושי

שלוש רמות הקושי מפעילות את אותו מנוע אלגוריתמי בסיסי, אך נבדלות ומובחנות ביניהן באמצעות שלושה פרמטרים מבניים קשיחים השולטים על עומק ורוחב הסריקה:

רמת קושי	אופק עומק החיפוש (Depth Horizon)	תקציב צמתים כולל (Node Budget)	רוחב הקרן החותך (K)	שכבות הגנה משולבות (Guard / Solver)
Easy	plies 2 (מהלכים קדימה)	100 צמתים	4 מצבים מובילים	ללא
Medium	plies 3 (מהלכים קדימה)	500 צמתים	8 מצבים מובילים	ללא
Hard	plies 4 (מהלכים קדימה)	2,000 צמתים	8 מצבים מובילים	מנגנון Capture Guard + פותר סיום Endgame Solver

אכיפת דטרמיניזם מלא: במטרה לעמוד במדויק בסעיף 7 ובהנחיות החובה של הנוהל הרשמי, מנוע החיפוש מנותק לחלוטין מרכיבי אקראיות או משעון המערכת (DateTime). עצירת החיפוש מבוקרת בצורה דטרמיניסטית קשיחה אך ורק על ידי ניצול תקציב הצמתים והגעה לאופק העומק המספרי. כתוצאה מכך, עבור אותה עמדת לוח נתונה, הסוכן יפיק תמיד את אותו המהלך בדיוק, ללא תלות בעומס המעבד או בחומרה עליה מורץ המשחק. דרגת הקושי נקבעת ומבוקרת מדעית אך ורק על ידי שינוי אופק הסריקה ועומק הראייה, ללא שימוש ברכיב אקראי הפסול על פי הנוהל.

9.11 מדוע האלגוריתם אינו מינימקס ועומד בהנחיות הנוהל?

סעיף 13 בנוהל אוסר במפורש שימוש באלגוריתמים הקלאסיים הממצים המוכרים: Minimax, Alpha-Beta Pruning, ו-MCTS. האסטרטגיה הנבחרת בפרויקט זה נבדלת מהם מבנית ומתמטית על פי הקריטריונים הבאים:

1. **היעדר שכבת מינימיזציה (No MIN Layer):** אלגוריתם מינימקס מפצל את העץ בתורו של היריב ובדוק את כל המהלכים החוקיים שלו מתוך הנחה שהוא יבחר במהלך בעל הציון הנמוך ביותר עבורנו (min). האלגוריתם הנוכחי אינו מבצע פיצול או מינימיזציה בתורו של היריב; הוא מחשב צעד יחיד חמדני צפוי מתוך מודל היריב, ובכך הוא מהווה פריסה חד-צדדית מכוונת מסלול ואינו נמנה עם משפחת המינימקס.

2. **האצלת ערכים חד-צדדית (Asymmetric Backup):** בשלב ה-Backup, המערכת אינה מחליפה פונקציות של $\max$ ו- $\min$ בין השכבות. בצמתי היריב, הערך המועלה מעלה שווה במדויק לערך הצאצא היחיד שנחזה במסלול, ללא הפעלת פונקציית מינימום.
3. **ניהול חזית מבוסס תור עדיפויות:** המינימקס סורק את המרחב בצורה עיוורת לפי סדר עומק (DFS) תוך שימוש במחסנית קריאות רקורסיבית מוחלטת. האלגוריתם הנוכחי מנהל את חזית החיפוש בצורה רוחבית מכוונת מידע באמצעות תור עדיפויות מנוהל (MinHeap).
4. **משקולות דינמיות מבוססות אוטומט:** נוסחת ההערכה במינימקס קלאסי היא קשיחה וקבועה. המערכת הנוכחית משנה ומחליפה את משקולות הנוסחה דינמית בזמן ריצה באמצעות מודל מכונת מצבים (FSA).
5. **עצירה מבוססת תקציב משאבים:** החיפוש עוצר ומבוקר על ידי ניצול תקציב צמתיים נוקשה, בניגוד לסריקה הממצה עד אופק מלא המאפיינת את המינימקס והאלפא-בטא.
6. **חוסר אקראיות מוחלט:** בניגוד לאלגוריתם MCTS (שנאסר) הנשען על סימולציות סטטיסטיות אקראיות (Playouts) ושעון זמן, הליבה הנוכחית היא דטרמיניסטית ומתמטית לחלוטין ומהווה פונקציות טהורות של מצב הלוח.
- גם פותר מצבי הסיום (EndgameSolver), למרות שהוא סורק את המרחב עד לעלי הסיום, אינו משתמש בשכבת $\min$ מלאה או בחישוב מינימקס קלאסי. הוא שומר על אותה ארכיטקטורה חד-צדדית המנבאת מהלך חמדני בודד של היריב, ולכן הוא חוקי לחלוטין ועומד במלואו בהנחיות החובה של הנוהל.

10. ארכיטקטורה של הפתרון (Top-down Level Design)

הפרויקט בנוי בארכיטקטורת הפרדת שכבות במודל MVVM, כנדרש בנוהל (סעיף 18). הפתרון מחולק כיום לארבעה פרויקטים עצמאיים ומבודדים בתוך Solution היררכי אחד (MancalaProject.sln): שכבת הליבה הלוגית, (MancalaProject.Core), שכבת הממשק הגרפי (MancalaProject.Wpf), שכבת ממשק הבדיקות המהירות (MancalaProject.Console), ופרויקט בדיקות היחידה האוטומטיות (MancalaProject.Tests).

לצורך solution explorer

כיוון התלות בין השכבות הוא חד-כיווני ונאכף באופן קשיח על-ידי המהדר:

- ✓ Core → Wpf ומכיר אותו.
- ✓ Console → Core ומכיר אותו.
- ✗ Core אינו תלוי באף שכבה אחרת ואינו מכיר אותן.

הפרדת השכבות אינה נשענת על משמעת המתכנת בלבד, אלא נאכפת על ידי כלי הפיתוח: פרויקט ה-Core מוגדר כפרויקט מסוג net8.0 (ולא net8.0-windows), ולכן אינו מסוגל לייבא רכיבי UI או מחלקות של WPF, ניסיון כזה יגרור שגיאת קומפילציה. שכבת ה-Model (שכבת ה-Core) אינה מכירה את שכבת התצוגה; מנוע המשחק מודיע על שינויים במצב הלוח באמצעות אירועים, ושכבת ה-ViewModel נרשמת אליהם ומגררת אותם לעדכוני ממשק דינמיים.

11. תרשים מקרי-שימוש (UML Use Cases)

בעקבות הוספת אפשרות הגדרת השחקן הפותח ומנגנון הניווט הפנימי, שודרג תרשים מקרי השימוש. מקרה השימוש הישן "בחירת רמת קושי" הורחב לכדי מקרה שימוש כולל בשם "הגדרת קבוצת המשחק", ונוסף מקרה שימוש עצמאי המאפשר חזרה למסך התפריט בעיצומו של קשר.

תמונה מdraw.ion

פירוט מקרי השימוש:

מקרה שימוש	תיאור
הגדרת קבוצת המשחק	במסך הפתיחה, המשתמש קובע את רמת הקושי של הסוכן (Easy / Medium / Hard) ובנוסף בוחר מי יבצע את המהלך הראשון (השחקן או המחשב).
התחלת משחק	יצירת חלון המשחק הראשי ופריסת הלוח במצב ההתחלתי. מקרה שימוש זה כולל בתוכו («include») את הגדרת מאפייני הקבוצה.
ביצוע מהלך	השחקן בוחר בור חוקי בצד שלו. המערכת מריצה את חוקי המשחק, ובמידה והתור הועבר - מפעילה אוטומטית את תגובת הסוכן.
חזרה למסך הפתיחה	לחיצה על כפתור "Setup" בעיצומו של המשחק; המערכת סוגרת את החלון הנוכחי ופותחת מחדש את מסך התפריט הראשי ללא סגירת האפליקציה.

התחלת משחק חדש	לחיצה על כפתור "New Game" מאפסת את הלוח הנוכחי ומחזירה אותו למצב ההתחלתי תחת אותן הגדרות קושי ויוזם פעילים.
צפייה בתוצאת המשחק	עם הגעת המשחק לסיומו הרשמי, מוצגת למשתמש הודעת סיכום חלונאית (ניצחון / הפסד / תיקו).

12. מבני נתונים

בהתאם להנחיות החובה של הנוהל (סעיף 10) האוסרות על שימוש בספריות חיצוניות מוכנות עבור הליבה האלגוריתמית, מבני הנתונים המרכזיים בפרויקט מומשו באופן עצמאי על ידי הסטודנט על מנת לתמוך במנוע החיפוש.

12.1 מערך הלוח

מצב הלוח מיוצג במלואו באמצעות מערך שלמים פשוט בגודל 14 ($\text{int}[14]$). חלוקת האינדקסים במערך מוגדרת כך:

- אינדקסים 0–5: ששת הבורות הרגילים של שחקן 1.
- אינדקס 6: המאגר (קופה) של שחקן 1.
- אינדקסים 7–12: ששת הבורות הרגילים של שחקן 2.
- אינדקס 13: המאגר (קופה) של שחקן 2.

ייצוג זה מאפשר גישה וקריאה בזמן קבוע של $O(1)$ לכל תא בלוח. בנוסף, הוא מאפשר ביצוע פעולת שכפול מהירה וזולה (Clone) של המערך החיונית להפעלת מנוע החיפוש, המעתיק ומנתח מאות מצבי לוח פוטנציאליים בכל תור.

12.2 תור עדיפויות – MinHeap (מבנה עצמי)

הפרונטיר (חזית החיפוש) מנוהל באמצעות ערימת מינימום בינארית שנכתבה מאפס. הערימה ממופה על גבי מערך דינמי רציף המייצג עץ בינארי שלם: עבור איבר הממוקם באינדקס i , ההורה שלו נמצא באינדקס $(i - 1) / 2$, וילדיו נמצאים באינדקסים $2i + 1$ ו- $2i + 2$.

פעולה	תיאור	סיבוכיות
Push	הכנסת איבר חדש למערך וביצוע הפעולה החישובית bubble-up	$O(\log n)$

שליפת איבר המינימום, העברת הרכיב האחרון לראש וביצוע bubble-down	Pop	$O(\log n)$
צפייה באיבר בעל העדיפות הגבוהה ביותר מבלי לשלוף אותו	Peek	$O(1)$

המבנה מומש בצורה גנרית ($\text{MinHeap} < TItem, TPriority >$) המאפשרת שימוש חוזר בטיפוסים שונים. על מנת לקבל התנהגות של ערימת מקסימום (הצפת הצמתים בעלי הציון ההיוריסטי הגבוה ביותר לראש התור), הצמתים מוכנסים לערימה עם עדיפות שלילית ($-h$). שגרות ה-bubble-up וה-bubble-down מומשו באמצעות רקורסיית-זנב (Tail-recursion) ולא בלולאות המכילות עצירת break, זאת במטרה לעמוד במדויק בקריטריונים המתודיים שנדרשו במחונן.

12.3 צומת החיפוש - SearchNode

כל מצב לוח שמתגלה במהלך סריקת המרחב נשמר באובייקט מסוג SearchNode. מחלקה זו מאגדת: את מצב הלוח הנוכחי, את מהלך-השורש שהוביל אליו, את עומק הצומת, את שלב המשחק, את הציון ההיוריסטי שלו ורשימת מצבים בנים (Children). אוסף הצמתים והבנים בונה את עץ החיפוש, שעליו מופעל מנגנון ה-backup בסיום הסריקה. הצומת אינו מחזיק מצביע ישיר לצומת ההורה שלו; שמירת "מהלך-השורש" בלבד מספקת ומאפשרת לשחזר את שרשרת ההחלטות שהובילה אליו ללא בזבז זיכרון.

12.4 טבלת טרנספוזיציות

במטרה למנוע חישוב חוזר והערכה היוריסטית כפולה עבור מצבי לוח זהים שהושגו דרך רצפי מהלכים שונים, מנוהלת טבלת גיבוב מסוג $\text{Dictionary} < \text{long}, \text{double} >$. המפתח מורכב מערך גיבוב המייצג את קונפיגורציית 14 התאים ביחד עם זהות השחקן הנוכחי, והערך הוא הציון ההיוריסטי המשויך לו. פעולות החיפוש וההכנסה מתבצעות בזמן ממוצע של $O(1)$. טבלה זו "מקפלת" את מבנה עץ החיפוש והופכת אותו לגרף מכון ללא-מעגלים (DAG), דבר החוסך משאבי חישוב יקרים.

12.5 אוטומט שלבי המשחק (FSA)

מבנה הנתונים המייצג את המצבים באוטומט מבוסס על הטיפוס enum GamePhase, המגדיר חמישה מצבים אפשריים: Opening, Midgame, Endgame, StarvationMode, ו-CaptureChain. פונקציית המעבר של האוטומט מנתחת את מאפייני הלוח ומחשבת את השלב הנוכחי בזמן קבוע של $O(1)$ על ידי סריקה מהירה של מערך 14 התאים הקבוע. האוטומט הוא חסר-זיכרון (Stateless): מצבו אינו נשמר אלא מחושב מחדש בכל פנייה בהתאם למצב הלוח הנתון.

12.6 רשומת תוצאת מהלך – MoveResult

טיפוס מסוג record המהווה טיפוס נתונים בלתי משתנה (Immutable) המשמש לריכוז תוצאות חישוב של מהלך בודד במנוע המשחק. הרשומה כוללת את אינדקס הבור שבו נחתה האבן

האחרונה, משתנה בוליאני המציין האם הוענק תור נוסף, ומשתנה בוליאני המציין האם התרחשה חטיפה. ערכי הרשומה נקבעים אך ורק בעת יצירת המופע (Init-only) ואינם ניתנים לשינוי לאחר מכן. הבחירה ב-record מבטיחה חוסר שינוי (Immutability) ומאפשרת להעביר את תוצאות המהלכים בין השכבות בצורה בטוחה, בניגוד לקבועי קומפילציה (const) של שפת #C החלים על טיפוסים פרימיטיביים בלבד ברמת המהדר.

12.7 מנגנון טבלת הזיכרון של פותר הסיום (EndgameSolver Memoization)

מחלקת ה-EndgameSolver מנהלת טבלת זיכרון ייעודית ומבודדת מסוג Dictionary<string, int>. תפקידה המבני הוא לאחסן את הערך המספרי המוחלט (הפרש אבני המאגר הסופי האמיתי) עבור כל מצב לוח ייחודי שפיצוחו הושלם במהלך פריסת עץ הסיום.

המפתח המשמש לפנייה לטבלה מיוצר כמחרוזת טקסט אחידה המקודדת במדויק את כמות האבנים ב-14 התאים, בצירוף מזהה ייחודי של זהות השחקן התורן (לדוגמה: "P1|4,4,4,4,4,4,0,4,4,4,4,4,0"). הציון המאוחסן כערך הוא מספר שלם המייצג את תוצאת ההכרעה הוודאית מאותה נקודה.

מנגנון זה מונע ביצוע חישובים ופריסות חוזרות של ענפים זהים שהושגו בסדר מהלכים שונה, "מקפל" את עץ הסיום לגרף מכון ללא מעגלים (DAG), ומבטיח כי כל עמדה ייחודית תיפתר פעם אחת בלבד לאורך הריצה. המפתח הטקסטואלי נבחר במכוון (במקום מפתח טיפוס long) כדי לאפשר למפתח לבצע הדפסת פלטי קריאה וניפוי שגיאות (Debugging) קלים ומהירים לאורך חקר מצבי הקצה, וכדי לחסום לחלוטין תופעות של התנגשויות גיבוב בשלב ההכרעה המתמטי המדויק. גודל הטבלה חסום קשיח על ידי תקציב הבטיחות של הרכיב (250,000 מצבים).

13. תיאור סביבת העבודה ושפת התכנות

רכיב	בחירה	תיאור ומטרת השימוש
שפת תכנות	#C	שפת הפיתוח המרכזית. שפה מונחית-עצמים המותרת לשימוש על פי סעיף 15 בנוהל.
שפת תיאור הממשק	XAML	שפת סימון (Declarative) המשמשת לעיצוב והגדרת פקדי הממשק הגרפי ב-WPF.

מסגרת עבודה	NET 8.0.	פלטפורמת הפיתוח המודרנית עליה נשענת המערכת כולה.
טכנולוגיית UI	WPF	Windows Presentation Foundation - הטכנולוגיה המשמשת לבניית הממשק החלונאי של הפרויקט.
סביבת פיתוח (IDE)	Visual Studio 2022	סביבת הפיתוח הרשמית ששימשה לכתובה, ניפוי שגיאות וקומפילציה (סעיף 17 בנוהל).
מבנה הפתרון	Solution אחד	קובץ פתרון יחיד (MancalaProject.sln) המאגד את שלושת הפרויקטים השונים.

מדוע נבחרו #C ו-WPF:

השימוש בשפת #C מאפשר ניהול מונחה-עצמים מתקדם, אכיפת טיפוסים קשיחה ותמיכה מלאה בהפרדת שכבות הנאכפת על ידי המהדר. פלטפורמת WPF נבחרה לפיתוח הממשק הגרפי בזכות מנגנון ה-Data Binding המובנה שלה, המאפשר קישור נתונים טבעי וישיר ותומך באופן מלא בארכיטקטורת MVVM הנדרשת על פי הנוהל.

הליבה האלגוריתמית ומנוע המשחק פותחו כפרויקט מסוג ספריית מחלקות (Class Library) ניטרלי לחלוטין (MancalaProject.Core). הפרדה זו מבטיחה שקוד הליבה אינו תלוי ברכיבי תצוגה כלשהם, ומאפשרת להריץ את המנוע באופן זהה הן תחת ממשק ה-WPF והן תחת פרויקט הקונסול המשמש להרצת בדיקות רגרסיה מהירות.

14. אלגוריתם ראשי (פסאודו-קוד)

סעיף זה מציג את הפסאודו-קוד הפורמלי של הליבה האלגוריתמית, החל מהקריאה הציבורית של הסוכן ועד להחזרת המהלך הנבחר לשכבת התצוגה. הזרימה מורכבת מארבעה בלוקים אלגוריתמיים מרכזיים: הפעולה הראשית (FindBestMove), לולאת חישוב הקרן (RunBeam), מעבר האצלת הערכים לאחור (Backup), ושכבת הסינון הקשיחה של שומר-ההגנה (ApplyCaptureGuard). תיאור מילולי מלא וטבלאות יעילות של מתודות אלו מופיעים בסעיף 9 ובסעיף 17.3.

מוסכמות סימון בקוד-הפסאודו: הסימן ← מסמן השמת ערך למשתנה.

● הסימן // מסמן הערת הסבר פנימית.

- קריאות לפונקציות ומתודות בעלות שם מוגדרות במבנה הסטנדרטי:
.(Name(parameters

14.1 הפונקציה הראשית – FindBestMove

פונקציה (FindBestMove (engine, difficulty, player):

קלט:

engine // מנוע המשחק ומצב הלוח הנוכחי
difficulty // דרגת הקושי הפעילה (Easy / Medium / Hard)
player // זהות הסוכן הממוחשב (Player1 / Player2)

פלט:

bestMove // אינדקס הבור הנבחר לביצוע המהלך

// שלב 1: בדיקת רשת ביטחון טרמינלית – פותר הסיום (בלעדי ל-Hard)
אם (difficulty == Difficulty.Hard) אז:

מהלך פותר ← 1-

אם (true == EndgameSolver.TrySolve(engine, player, out)) אז:

החזר מהלך פותר // נמצא פתרון מתמטי מוחלט, מדלגים על ה-Beam

// שלב 2: טעינת פרמטרי החיפוש הדטרמיניסטיים לפי רמת הקושי (עומקי 2/3/4 plies)

(horizon, totalBudget, K) ← GetSearchParameters(difficulty)

// Easy: (2, 100, 4)

// Medium: (3, 500, 8)

// Hard: (4, 2000, 8)

// שלב 3: חלוקת תקציב שוויונית בין מהלכי השורש ובניית תת-העצים

מהלכי שורש ← engine.GetValidMoves()

budget פנימי לכל מהלך ← totalBudget / Length (מהלכי שורש)

rootChildren ← אתחל רשימה ריקה

לכל move בתוך מהלכי שורש תעשה:

childEngine ← engine.Clone()

childEngine.ApplyMove(move)

phase ← PhaseAutomaton.Detect(childEngine, player)

h ← Heuristic.Evaluate(childEngine, player, phase)

```

node ← SearchNode(childEngine, rootMove: move, depth: 1, h, phase)
      K, horizon, player), RunBeam(node, budget
      (node.Value ← Backup(node, player
      rootChildren אל רשימת node אוסף

```

```

// שלב 4: הפעלת שגרת דירוג ובחירת הצומת המוביל בעל ערך ה-Backup המרבי
bestNode ← Argmax(rootChildren, key: node → node.Value)
chosenMove ← bestNode.RootMove

```

```

// שלב 5: הפעלת שכבת סינון קשיחה – שומר ההגנה (בלעדי ל-Hard)
אם (difficulty == Difficulty.Hard) אז:

```

```

chosenMove ← ApplyCaptureGuard(engine, rootChildren, player,
                                (chosenMove

```

החזר chosenMove
סיום פונקציה

14.2 לולאת חישוב הקרן – RunBeam

:(RunBeam (rootNode, budget, K, horizon, agentPlayer

קלט:

```

rootNode // צומת השורש של תת-העץ הנוכחי
budget // תקציב המצבים המקסימלי הניתן לפריסה בתת-עץ זה
K // רוחב הקרן החותר (כמות הצאצאים המובילים שיישמרו)
horizon // אופק העומק המקסימלי של הסריקה במספר plies
agentPlayer // זהות הסוכן הממוחשב

```

```

frontier ← יצירת MinHeap חדש // תור עדיפויות מנוהל (שמירה עם עדיפות שלילית: -H)
transpositionTable ← אתחול מילון חדש מסוג Dictionary<long, double>
nodesExpanded ← 0

```

```

(frontier.Push(rootNode, priority: -rootNode.H

```

כל עוד (nodesExpanded < budget וגם frontier.IsEmpty == false) תעשה:

```

current ← frontier.Pop() // שליפת הצומת בעל הציון ההיוריסטי הגבוה ביותר מהערימה
nodesExpanded ← nodesExpanded + 1
// שלב בדיקת תנאי עצירה (הגעה לאופק עומק או למצב טרמינלי)
אם (current.State.IsGameOver == true או current.Depth >= horizon) אז:

```

המשך לאיטרציה הבאה // הצומת מוגדר כעלה, ערכו מחושב ישירות ב-Backup ולא מורחב

// בדיקת איחוד מצבים בגרף (טבלת טרנספוזיציות מבוססת FNV-1a)
(key ← HashFNV1a(current.State, current.NextPlayer)
אם (transpositionTable.Contains(key) == true) אז:

[current.H ← transpositionTable[key]
המשך לאיטרציה הבאה // נמנע ביצוע חישוב היוריסטי חוזר, ענף זה ממוזג בגרף
אחרת:
transpositionTable[key] ← current.H

// פריסת הצומת והרחבתו בהתאם לזהות השחקן התורן
children ← רשימה ריקה
אם (current.NextPlayer == agentPlayer) אז:

(children ← ExpandAgentNode(current, K, agentPlayer)
אחרת:
(children ← ExpandOpponentNode(current, agentPlayer

לכל child בתוך children תעשה:
current.Children.Add(child)
(frontier.Push(child, priority: -child.H
סיום פונקציה

פונקציה (ExpandAgentNode (parent, K, agentPlayer):

candidates ← רשימה ריקה
מהלכים חוקיים ← (parent.State.GetValidMoves
לכל move בתוך מהלכים חוקיים תעשה:

(childEngine ← parent.State.Clone
תוצאה ← (childEngine.ApplyMove(move
// מנגנון כוונת תורות נוספים: במידה וחוק המשחק העניק תור נוסף לסוכן, פונקציית
ה-ApplyMove מריצה לולאת פיזורים פנימית עד להעברת התור, ובכך הרצף כולו מכוון לכדי
קשת בודדת באוטומט.

phase ← PhaseAutomaton.Detect(childEngine, agentPlayer)
h ← Heuristic.Evaluate(childEngine, agentPlayer, phase)
child ← SearchNode(childEngine, parent.RootMove, parent.Depth + 1, h,
phase)
הוסף את child אל רשימת candidates

// שלב חיתוך הקרן (Beam Pruning): דירוג ושמירת הצאצאים המבטיחים ביותר
מיין את רשימת candidates בסדר יורד לפי ערך H
החזר את K האיברים הראשונים מתוך רשימת candidates
סיום פונקציה

פונקציה (ExpandOpponentNode (parent, agentPlayer):

// מודל היריב החמדן (Adversarial Greedy): חיזוי מהלך בודד ללא פריסת שכבת MIN
 מהלך יריב ← SimulateGreedyOpponentMove(parent.State)
 childEngine ← parent.State.Clone
 (מהלך יריב אינו 1-אז):

childEngine.ApplyMove (מהלך יריב)
 phase ← PhaseAutomaton.Detect(childEngine, agentPlayer)
 h ← Heuristic.Evaluate(childEngine, agentPlayer, phase)
 SearchNode(childEngine, parent.RootMove, parent.Depth + 1, h, ← child שני
 (phase
 החזר רשימה המכילה את child שני כרכיב יחיד
 סיום פונקציה

14.3 רובד האצלת הערכים לאחור – Backup

פונקציה (Backup (node, agentPlayer):

קלט:

node // צומת נוכחי בעץ החיפוש
 agentPlayer // זהות הסוכן הממוחשב

פלט:

value // הערך המספרי המואצל לאחור

// מקרה בסיס: הגעה לצומת עלה (צומת שלא הורחב או שהגיע לאופק העומק)
 אם (node.Children היא ריקה) אז:

אם (node.State.IsGameOver == true) אז:

// קיצור דרך טרמינלי: חישוב הפרש הניקוד האמיתי והכפלתו בגורם קשיח
 scoreDiff ← node.State.Store(agentPlayer) - node.State.Store
 שחקן(agentPlayer)
 החזר scoreDiff * 10000
 אחרת:

החזר node.H // החזרת הערך ההיוריסטי הישיר של העמדה

// מקרה רקורסיבי: מעבר על צאצאי הצומת ואיסוף ערכיהם המיועדים
 childValues ← רשימה ריקה
 לכל child בתוך node.Children תעשה:

חוסף את Backup(child, agentPlayer) אל רשימת childValues

אם (node.NextPlayer == agentPlayer) אז:

// שכבת הבחירה של הסוכן (MAX) – הנחת בחירה במסלול המיטבי

החזר Max(childValues)

אחרת:

// שכבת החיזוי של היריב – העלאת ערך הילד היחיד שנחזה במסלול, ללא פריסת MIN

החזר childValues[0]

סיום פונקציה

14.4 שכבת הסינון הקשיחה – ApplyCaptureGuard

פונקציה ApplyCaptureGuard (engine, rootChildren, agentPlayer, searchMove):

קלט:

// searchMove – אינדקס המהלך שהמנוע של ה-Beam בחר

// agentPlayer – זהות הסוכן הממוחשב

// rootChildren – רשימת צמתי השורש שנסרקו; כל אחד נושא RootMove וערך Backup

// engine – מנוע המשחק ומצב הלוח המקורי לפני ביצוע המהלך

פלט:

// אינדקס המהלך הסופי שיבוצע (המהלך המקורי או חלופת הגנה)

קבועים:

// CaptureGuardMargin <- 3 חלופה חייבת להיות בטוחה ב-3 אבנים לפחות כדי

להצדיק עקיפה

// CaptureGuardMinExposure <- 3 חטיפת יריב הקטנה מ-3 אבנים נחשבת זניחה

ומותרת

לוגיקה מבצעת:

// שלב 1: כמה אבנים יחטוף היריב בתגובתו אם נשחק את מהלך ה-Beam?

חשיפת המהלך <- OpponentMaxCaptureAfter(engine, searchMove, agentPlayer)

// שלב 2: אם החשיפה זניחה, או שהמהלך עצמו מבצע חטיפה (סחר מכוון ולא טעות) —

אשר אותו

אם (חשיפת המהלך > CaptureGuardMinExposure או CaptureCountOf(engine,

searchMove) > 0 אז:

החזר searchMove

// שלב 3: מצא את החשיפה הקטנה ביותר שמהלך שורש חוקי כלשהו מאפשר ליריב
חשיפה מינימלית <= MinExposureOverRootMoves(engine, rootChildren, agentPlayer)
(agentPlayer)

// שלב 4: אם אין מהלך בטוח משמעותית יותר — החשיפה בלתי נמנעת, השאר את בחירת
המנוע
אם (חשיפת המהלך - חשיפה מינימלית > CaptureGuardMargin) אז:
החזר searchMove

// שלב 5: קיימת חלופה בטוחה בבירור — החזר את המהלך הבטוח בעל ערך ה-Backup
הגבוה ביותר
החזר SafestRootMoveByValue(agentPlayer, rootChildren, engine, חשיפה
מינימלית)
סיום פונקציה

פונקציה OpponentMaxCaptureAfter (engine, move, agentPlayer):

// כמות האבנים המרבית שהיריב יכול לחטוף בתגובתו המיידית אם הסוכן ישחק את move
עותק -> engine.Clone()
עותק.ApplyMove(move)
אם (עותק.IsGameOver() == true או עותק.CurrentPlayer == agentPlayer) אז:
// המהלך סיים את המשחק או שמר את התור לסוכן — אין ליריב תגובה
החזר 0

חשיפה מרבית -> 0

כל עוד בתוך קריאת עותק.GetValidMoves() קיים מהלך-יריב עשה:
נחטף <- CaptureCountOf(עותק, מהלך-יריב)
אם (נחטף < חשיפה מרבית) אז:
חשיפה מרבית -> נחטף
החזר חשיפה מרבית
סיום פונקציה

פונקציה (CaptureCountOf (state, move):

// כמות האבנים שנחטפות אם משחקים את move מ-state
עותק -> state.Clone()
החזר עותק.CapturedStoneCount.ApplyMove(move).
סיום פונקציה

פונקציה (MinExposureOverRootMoves (engine, rootChildren, agentPlayer):

// החשיפה הקטנה ביותר שמהלך שורש חוקי כלשהו מאפשר ליריב
מינימום -> אינסוף
כל עוד בתוך rootChildren קיים root עשה:

חשיפה -> OpponentMaxCaptureAfter(engine, root.RootMove, agentPlayer)
אם (חשיפה > מינימום) אז:
מינימום -> חשיפה
החזר מינימום
סיום פונקציה

פונקציה (SafestRootMoveByValue (agentPlayer, rootChildren, engine, חשיפה מינימלית):

// מבין מהלכי השורש שחושפים רק את המינימום – מחזירה את בעל ערך ה-Backup הגבוה ביותר
הכי טוב -> rootChildren[0]
ערך מיטבי -> SafeRootValue(agentPlayer, הכי טוב, engine, חשיפה מינימלית)
כל עוד בתוך rootChildren החל מהאיבר השני קיים root עשה:
ערך -> SafeRootValue(agentPlayer, root, engine, חשיפה מינימלית)
אם (ערך < ערך מיטבי) אז:
ערך מיטבי -> ערך
הכי טוב -> root
החזר הכי טוב.RootMove.
סיום פונקציה

פונקציה (SafeRootValue (agentPlayer, root, engine, חשיפה מינימלית):

// ערך ה-Backup של מהלך השורש אם הוא בין הבטוחים ביותר, אחרת מינוס אינסוף –
 // כך ש-SafestRootMoveByValue לעולם לא יבחר מהלך לא בטוח
 אם (OpponentMaxCaptureAfter(engine, root.RootMove, agentPlayer) => חשיפה מינימלית) אז:
 החזר (root.Backup(agentPlayer)
 אחרת:
 החזר מינוס אינסוף
 סיום פונקציה

14.5 פותר מצבי הסיום – EndgameSolver.TrySolve

פונקציה (TrySolve (engine, player, out bestMove):

קלט:

engine // מנוע המשחק ומצב הלוח המקורי לפני ביצוע המהלך
 player // זהות השחקן הנוכחי (הסוכן החכם)

פלט:

bestMove // אינדקס המהלך המנצח המדויק (מוחזר כפרמטר פלט)
 ערך מוחזר בוליאני // true אם נמצא פתרון בתקציב, false אחרת

bestMove ← -1

סך אבנים ← סך כל האבנים ב-12 הבורות הרגילים בלוח

// הפותר כשיר לפעולה רק בשלב הסיום הרשמי של המשחק
 אם (סך אבנים => 12 או (engine.IsGameOver == true) אז:

החזר false

memo ← אתחל מילון גיבוב ריק מסוג Dictionary<string, int>

nodeCount ← 0

מהלכים חוקיים ← engine.GetValidMoves()

ערך מיטבי ← מינוס אינסוף

לכל מהלך בתוך מהלכים חוקיים תעשה:

עותק מדמה ← engine.Clone()

עותק מדמה.ApplyMove(מהלך)

IsAgentTurn ← (עותק מדמה.player == CurrentPlayer)

ערך מחושב ← SolveRecursive(עותק מדמה, memo, player, IsAgentTurn)

(nodeCount

// הגנה מפני תקיעת הממשק – אם חרגנו מתקציב הבטיחות, הפותר קורס ומבצע נסיגה

ל-Beam
אם (nodeCount > 250000) אז:

החזר false

אם (ערך מחושב < ערך מיטבי) אז:

ערך מיטבי ← ערך מחושב
bestMove ← מהלך

החזר true
סיום פונקציה

14.6 הרקורסיה הפנימית של פותר הסיום – EndgameSolver.Solve

פונקציה (Solve (engine, player, isAgentTurn, memo, nodeCount):

קלט:

engine // מנוע המשחק ומצב הלוח המקורי
player // זהות השחקן הנוכחי (הסוכן החכם)
isAgentTurn // האם זהו תורו של הסוכן החכם ברקורסיה
memo // טבלת הזיכרון (מילון הגיבוב המשותף)
nodeCount // מונה פריסת הצמתים לבקרת בטיחות (מועבר כהפניה)

פלט:

ערך מוחזר ← הפרש הניקוד הטרמינלי המוחלט

nodeCount ← nodeCount + 1
אם (nodeCount > 250000) אז:

החזר 0

אם (engine.IsGameOver == true) אז:

engine.FinalizeBoard()
הפרש סופי ← engine.GetScore(player) - engine.GetScore((player))
החזר הפרש סופי // החזרת תוצאה מספרית מוחלטת של הניקוד בסוף המשחק

מפתח מצב ← BuildKey(engine, isAgentTurn)
אם (מפתח מצב קיים בתוך memo) אז:

החזר memo[מפתח מצב] // שליפה מהירה בזמן של O(1) ומניעת פריסה כפולה

ערך מחושב ← 0
אם (isAgentTurn == true) אז:

// שכבת הבחירה של הסוכן (MAX) – מקסום הרווח מבין כל ההמשכים החוקיים

ערך מרבי ← מינוס אינסוף

מהלכים ← engine.GetValidMoves()

לכל מהלך בתוך מהלכים תעשה:

עותק ← engine.Clone()

עותק.ApplyMove(מהלך)

האם תור שלי ← (עותק.player == currentPlayer)

ערך מרבי ← Max(ערך מרבי, Solve(עותק, player, האם תור שלי, memo, nodeCount))

ערך מחושב ← ערך מרבי

אחרת:

// שכבת החיזוי של היריב – סימולצית צעד בודד חמדני מתוך המודל המשותף

מהלך יריב ← engine.SimulateGreedyOpponentMove(BeamSearch)

עותק ← engine.Clone()

אם (מהלך יריב אינו 1-אז:

עותק.ApplyMove(מהלך יריב)

האם תור שלי ← (עותק.player == currentPlayer)

ערך מחושב ← Solve(עותק, player, האם תור שלי, memo, nodeCount)

memo[מפתח מצב] ← ערך מחושב // שמירת הפתרון הטרמינלי המדויק בגרף ה-DAG

החזר ערך מחושב

סיום פונקציה

הערות מתודיות מובנות:

- מתודות העזר ופרוצדורות המשנה הפנימיות כגון, PhaseAutomaton.Detect, TrySolve מפורטות ומיוצגות כסעיפים עצמאיים תחת תתי-הסעיפים של פרק 17 (סעיפים 17.3, 17.4, 17.5, 17.9-ו בהתאמה).
- מבנה נתונים תור העדיפויות MinHeap ושגרות העזר הרקורסיביות שלו (Push, Pop, BubbleUp, BubbleDown) מתועדים במלואם תחת סעיף 12.2 וסעיף 17.6.
- **ניתוח סיבוכיות כולל:** סיבוכיות זמן הריצה הכוללת של המתודה FindBestMove on קשיחה קבועה על ידי הנוסחה $O(\text{totalBudget} \cdot \log(\text{totalBudget}))$ כאשר N_{endgame} שווה ל-0 עבור רמות הקושי Easy ו-Medium וחסום קשיח עד 250,000 מצבים עבור פותר הסיום ברמת קושי Hard. במונחים מעשיים של מחשב אישי, קריאה ראשית למתודה מסתיימת בטווח של מאיות השנייה ועד מאות מילישניות בודדות, ללא תלות בעומס המעבד או בחומרה.

15. תיאור ממשקים חיצוניים

15.1 מסך הפתיחה (SetupWindow)

המסך הנפתח עם הרצת התוכנה, המאפשר למשתמש לקבוע את מאפייני הרצה של מחזור המשחק:

תמונה

- המשתמש קובע את רמת הקושי ואת זהות השחקן שיבצע את המהלך הראשון (יוזם המשחק).
- לחיצה על **Start** סוגרת את מסך הפתיחה ופותחת את חלון המשחק בהתאם לבחירה.

15.2 חלון המשחק הראשי (MainWindow)

כותרת חלון המשחק שודרגה ומכילה כעת שני כפתורי שליטה וניווט בפינה הימנית העליונה:

תמונה

- **כפתור Setup:** סוגר את חלון המשחק הנוכחי ומחזיר את המשתמש באופן מיידי אל מסך הפתיחה הראשי לצורך שינוי הגדרות קושי או החלפת יוזם, מבלי לסגור את ריצת התוכנה.
- **כפתור New Game:** מאתחל משחק חדש באותן הגדרות קושי ויוזם פעילים.

15.3 מנגנון הקישור (Data Binding)

מאפייני הבחירה של רמת הקושי והשחקן המתחיל מקושרים באמצעות Data Binding ישירות ל-SetupViewModel. כפתור ה-**Setup** החדש בחלון המשחק מנוהל ומטופל ישירות מתוך קוד החלון (Code-Behind) של שכבת התצוגה, שכן יצירת חלונות וניווט ביניהם הם באחריותה הבלעדית של שכבת ה-UI ואינם מבוצעים דרך שכבת ה-Model.

16. תרשים מחלקות (UML Class Diagram)

סעיף זה מפרט את רכיבי המחלקות, טיפוסיהן, והקשרים המבניים ביניהם. פירוט זה מהווה את בסיס הנתונים לצורך שרטוט דיאגרמת ה-UML הגרפית בכלי ייעודי (כגון draw.io) להטמעה בתיק.

16.1 מחלקות שכבת ה-Core

מחלקה / סוג	טיפוס	חברים ומתודות עיקריים
Player	enum	Player1, Player2
Difficulty	enum	Easy, Medium, Hard
GamePhase	enum	Opening, Midgame, Endgame, StarvationMode, CaptureChain
MoveResult	record	LastPitIndex, ExtraTurn, CaptureOccurred, CapturedFromPit, CapturedStoneCount
GameEngine	class	Board, CurrentPlayer; ApplyMove(), IsValidMove(), GetValidMoves(), IsGameOver(), ()FinalizeBoard(), GetWinner(), Clone
GreedyAgent	class	()CalculateMove(), ExecuteMove
BeamSearch	static class	()FindBestMove
EndgameSolver	static class	()TrySolve(), SolveRecursive(), BuildKey

(Evaluate)	static class	Heuristic
(Detect)	static class	PhaseAutomaton
Push(), Pop(), Peek(), Count	class (גנרי)	MinHeap<TItem, <TPriority
State, RootMove, Depth, H, Phase, Children	class	SearchNode

16.2 מחלקות שכבת ה-WPF

מחלקה	טיפוס	חברים ומתודות עיקריים
ObservableObject	abstract class	מימוש INotifyPropertyChanged; מתודות (OnPropertyChanged(), SetField
RelayCommand	class	מימוש ICommand; מתודות, Execute(), (CanExecute
RelayCommand	class (גנרי)	מימוש ICommand עבור טיפוסים מועברי פרמטרים; מתודות Execute(), CanExecute
GameViewModel	class	מאפייני הלוח (Pit0–Pit5, Pit7–Pit12, Store1, Store2), StatusMessage PlayPitCommand, NewGameCommand

משתנים בוליאניים לרדיו-באטנס (IsEasy, IsMedium, IsHard), המאפיין SelectedDifficulty	class	SetupViewModel
מחלקות הממשק הפיזיות ונקודת הכניסה לאפליקציה.	class	MainWindow, SetupWindow, App

16.3 קשרי הגומלין בין המחלקות (קישורים לדיאגרמה)

● הכלה ושימוש (→):

- GameViewModel מכילה ומפעילה אובייקט מסוג GameEngine ואובייקט מסוג GreedyAgent.
- GameViewModel מכילה ומקשרת פקודות מסוג RelayCommand ו-RelayCommand<int>.
- GreedyAgent מאצילה סמכויות ומפעילה את מחלקת החיפוש הסטטית BeamSearch.
- המחלקה הסטטית BeamSearch יוצרת ומשתמשת במבנים: MinHeap, SearchNode, PhaseAutomaton, Heuristic.
- המחלקה הסטטית EndgameSolver מופעלת על ידי BeamSearch (ברמת קושי Hard בלבד); היא משתמשת ב-GameEngine לסימולציות, קוראת למתודה BeamSearch.SimulateGreedyOpponentMove לחיזוי מהלך היריב, ומנהלת טבלת זיכרון (Memoization) מסוג Dictionary<string, int>.
- הצומת SearchNode מכיל אובייקטים מסוג GameEngine ומשתנה מצב מסוג GamePhase.
- **הרכבה עצמית בינארית (רקורסיבית):** מחלקת SearchNode מכילה רשימה של אובייקטים מסוג SearchNode (Children), המייצגת את ענפי המשך החיפוש.
- PhaseAutomaton משתמשת ומחזירה טיפוס מסוג GamePhase על בסיס ניתוח ה-GameEngine.
- המחלקה הסטטית Heuristic מנתחת את ה-GameEngine בהתאם ל-GamePhase וזהות ה-Player.
- GameEngine מחזירה תווים ומעדכנת באמצעות רשומות מסוג MoveResult ומשתמשי Player.

● זרימת האפליקציה הניהולית:

- המחלקה הראשית App מייצרת את החלון SetupWindow ומצמידה לו את ה-SetupViewModel.
- עם אישור התחלת המשחק, App מייצרת את החלון MainWindow ומצמידה לו את ה-GameViewModel.

17. תיאור המחלקות והפונקציות הראשיות

סעיף זה מספק פירוט מעמיק של המבנה הפנימי ומנגנוני הפעולה של מחלקות שכבת ה-Core (הליבה הלוגית של המערכת). לכל מחלקה מצורפת טבלת רכיבים המפרטת את המתודות החשובות, הפרמטרים, ערכי ההחזר וסיבוכיות זמן הריצה שלהן. עבור פונקציות בעלות משמעות אלגוריתמית מרכזית, מצורף גם תיאור מפורט באמצעות פסאודו-קוד.

17.1 מחלקת מנוע המשחק (GameEngine)

מחלקה זו מהווה את מנוע המשחק הרשמי והיא אחראית על ניהול מצב הלוח, אכיפת חוקי משחק המנקלה (בגרסת Kalah), ביצוע ולידציה למהלכים, וזיהוי מצבי סיום משחק. המחלקה מבודדת לחלוטין ואינה תלויה ברכיבי ממשק משתמש (UI).

פונקציה	פרמטרים	מחזירה	תפקיד ופעולה	סיבוכיות
ApplyMove	pitIndex: int	MoveResult	מבצעת מהלך שלם הכולל את שלבי הזריעה, החטיפה, בדיקת תור נוסף, החלפת שחקן תורן ושידור אירועים למערכת.	$O(s)$ כאשר s הוא מספר האבנים שנזרעו
IsValidMove	pitIndex: int	bool	בודקת האם המהלך המבוקש חוקי עבור השחקן הנוכחי (האם האינדקס שייך לצד שלו והבור אינו ריק).	$O(1)$
GetValidMoves	—	<List<int	מחזירה רשימה של כל אינדקסי הבורות החוקיים לביצוע מהלך	$O(1)$ (סיום ב-6 בורות)

	עבור השחקן הנוכחי.			
$O(1)$ (חסום ב-12 בורות)	בודקת האם תנאי הסיום הפיזי מתקיים (האם ששת הבורות של אחד הצדדים התרוקנו לחלוטין).	bool	—	IsGameOver
$O(1)$ (חסום ב-12 בורות)	אוספת את כל האבנים שנותרו בבורות הרגילים אל המאגרים של השחקנים המתאימים ומסמנת את סיום המשחק.	void	—	FinalizeBoard
$O(1)$	מחזירה את תוצאת המשחק לאחר הפעלת FinalizeBoard (ערכים: 1 לשחקן 1, -1 לשחקן 2, 0 לתיקו).	int	—	GetWinner
$O(1)$ (חסום ב-14 תאים)	מייצרת עותק עצמאי ומנותק (Deep Copy) של מנוע המשחק ומצב הלוח הנוכחי, לשימוש בסימולציות חיפוש.	GameEngine	—	Clone

פונקציות עזר מהירות לצורך גישה, קריאת נתונים וחישוב אינדקסים מקבילים על גבי הלוח.	int	לפי העניין	GetScore / GetPitIndex / GetOppositePitIndex	O(1)
---	-----	------------	---	------

פסאודו-קוד עבור המתודה ApplyMove:

קלט:

move // אינדקס הבור (0-5 עבור שחקן 1, 6-12 עבור שחקן 2) הנבחר לפיזור

פלט:

result // רשומת MoveResult המכילה את מאפייני סיום המהלך

לוגיקה מבצעת:

אבנים לפיזור ← Board[move]

Board[move] ← 0

תא נוכחי ← move

כל עוד (אבנים לפיזור > 0) תעשה:

אבנים לפיזור = Board[move]

Board[move] = 0

תא נוכחי = move

כל עוד (אבנים לפיזור > 0) עשה:

תא נוכחי = NextSowingIndex (תא נוכחי) // פונקציית עזר המדלגת על מאגר היריב

Board[תא נוכחי] = Board[תא נוכחי] + 1

אבנים לפיזור = אבנים לפיזור - 1

// בדיקת חוק תור נוסף (האבן האחרונה נחתה במאגר השחקן)

תור נוסף ← (תא נוכחי הוא המאגר של השחקן הנוכחי)

חטיפה התרחשה ← false

אבנים שנחטפו ← 0

// בדיקת חוק החטיפה (האבן האחרונה נחתה בתא ריק בצד שלי, ומולו יש אבני יריב)

אם (תא נוכחי הוא בור רגיל בשליטת השחקן הנוכחי וגם Board[תא נוכחי] == 1) אז:

תא נגדי ← 12 - תא נוכחי

אם (Board[תא נגדי] > 0) אז:

אבנים שנחטפו $\leftarrow \text{Board}[\text{תא נגדי}] + 1$
 $\text{Board}[\text{המאגר של השחקן הנוכחי}] \leftarrow \text{Board}[\text{המאגר של השחקן הנוכחי}] + \text{אבנים שנחטפו}$
 $\text{Board}[\text{תא נוכחי}] \leftarrow 0$
 $\text{Board}[\text{תא נגדי}] \leftarrow 0$
 חטיפה התרחשה $\leftarrow \text{true}$
 אם (תור נוסף $\text{false} ==$) אז:
 $\text{CurrentPlayer} \leftarrow \text{החלף שחקן}(\text{CurrentPlayer})$

החזר MoveResult (תא נוכחי, תור נוסף, חטיפה התרחשה, אבנים שנחטפו)
 סיום פונקציה

פסאודו-קוד עבור המתודה NextSowingIndex

NextSowingIndex (מאינדקס):
 הבא = (מאינדקס + 1) מודולו 14
 אם (הבא הוא המאגר של היריב) אז:
 החזר (הבא + 1) מודולו 14 // דילוג על מאגר היריב
 אחרת:
 החזר הבא
 סיום פונקציה

פסאודו-קוד עבור המתודה FinalizeBoard :

לוגיקה מבצעת:
 אם ($\text{isGameOver} == \text{true}$) אז:
 לכל בור בשורה של שחקן 1 תעשה:
 $\text{Board}[\text{מאגר שחקן 1}] \leftarrow \text{Board}[\text{מאגר שחקן 1}] + \text{Board}[\text{בור}]$
 $\text{Board}[\text{בור}] \leftarrow 0$
 לכל בור בשורה של שחקן 2 תעשה:
 $\text{Board}[\text{מאגר שחקן 2}] \leftarrow \text{Board}[\text{מאגר שחקן 2}] + \text{Board}[\text{בור}]$
 $\text{Board}[\text{בור}] \leftarrow 0$
 סיום פונקציה

17.2 מחלקת הסוכן החכם (GreedyAgent)

מחלקה זו משמשת כרכיב ה-Facade (תבנית עיצוב פאסאד) ומציגה את ה-API הציבורי של השחקן הממוחשב כלפי שכבת ה-ViewModel. היא אינה מכילה את לוגיקת החיפוש בעצמה אלא שומרת את נתוני הגדרת הראייה (זהות השחקן ורמת הקושי) ומאצילה את ביצוע החישוב למחלקה האלגוריתמית BeamSearch.

פונקציה	פרמטרים	מחזירה	תפקיד ופעולה	סיבוכיות
GreedyAgent	player: Player, difficulty: Difficulty	—	בנאי המחלקה; מאתחל סוכן ממוחשב ומקבע את זהותו ואת רמת הקושי שלו.	$O(1)$
CalculateMove	engine: GameEngine	int	מפעילה את מנוע החיפוש הסטטי ומחשבת את אינדקס המהלך המיטבי שנבחר, מבלי לשנות את מצב המנוע הנתון.	תלוי ברמת הקושי (ראה סעיף 17.3)
ExecuteMove	engine: GameEngine, move: int	MoveR esult	מבצעת ומחילה בפועל את המהלך שנבחר על גבי מנוע המשחק הראשי.	$O(s)$ (בהתאם ל-ApplyMove)

הסבר מבני: ההפרדה המתודית בין פונקציית החישוב (CalculateMove) לפונקציית הביצוע (ExecuteMove) חיונית עבור ארכיטקטורת המערכת; היא מאפשרת לשכבת התצוגה להציג משוב ויזואלי מהיר (כגון אנימציה או חיווי) על המהלך שנבחר, לפני שהוא מבוצע ומשנה את מצב הלוח הרשמי.

17.3 מחלקת מנוע החיפוש (BeamSearch)

מחלקה סטטית ופנימית בשם `internal static class BeamSearch`, המהווה את עמוד התווך האלגוריתמי של הסוכן הממוחשב לצורך ביצוע חישובים אסטרטגיים מקורבים בזמן אמת. המחלקה מעוצבת כרכיב חסר-מצב (Stateless), שבו כל תשתיות הנתונים, המערכים וצמתי חזית החיפוש מוקצים ומנוהלים על גבי מחסנית הקריאות בכל פנייה מחדש, מה שמבטיח הגנה מוחלטת מפני זליגת מידע בין חישובים שונים בריצה.

טבלת המתודות של מחלקת `BeamSearch`:

פונקציה	פרמטרים	ערך מוחזר	מתודה מבצעת (תפקיד ופעולה)	סיבוכיות זמן ריצה
FindBestMove	engine: GameEngine, difficulty: Difficulty, player: Player	int	נקודת הכניסה הציבורית של מנוע החיפוש. בודקת תחילה את פותר הסיום, ואם הלוח טרם נפתר מתמטית — מאתחלת את פרמטרי הקושי, מחלקת את תקציב המצבים שוויונית, מריצה את החיפוש ומפעילה את ה-Capture Guard.	חסומה קשיחה על ידי תקציב המצבים של רמת הקושי $O(1)$ ברמת המאקרו)
RunBeam	root: SearchNode, budget: int, K: int, horizon: int	void	מנהלת את פריסת עץ המשחק של אלגוריתם ה-Best-First Beam Search עבור מהלך שורש ספציפי אחד, תוך עדכון חזית ה-MinHeap וטבלת הטרנספוזיציות.	$O(N \cdot \log N)$ כאשר N הוא תקציב הצמתים המוקצה

Backup	node: SearchNode	double	מעבר רקורסיבי על גבי עץ החיפוש שנסרק, המחשב ומעלה לאחור (האצלה) את ערך הניקוד המיועד של הצומת על בסיס ערכי צאצאיו על פי כללי המודל.	$O(N \cdot N)$ כאשר N הוא סך כל הצמתים שיוצרו בעץ החיפוש
SimulateGreedy OpponentMove	engine: GameEngine	int	שגרת עזר פנימית המסמלצת ומחזירה את המהלך המקומי המבטיח ביותר עבור היריב (על בסיס רווח מיידית) לצורך בניית מסלול החיזוי הישיר.	$O(1)$ (חסום ב-6) בדיקות והערכות (קבועות)
ApplyCaptureGuard	engine: GameEngine, rootChildren: List<SearchNode>, myPlayer: Player, searchMove: int	int	שכבת סינון קשיחה בלעדית לרמת Hard. בודקת את חשיפת המהלך שנבחר, ואם מתגלה הפקרת חטיפה גדולה ליריב — מבצעת עקיפה ומחזירה מהלך אלטרנטיבי מגן.	$O(M^2)$ כאשר M הוא מספר המהלכים החוקיים בשורש

פסאודו-קוד עבור המתודה FindBestMove:

(FindBestMove (engine, difficulty, player

קלט:

engine // מנוע המשחק ומצב הלוח הנוכחי
difficulty // דרגת הקושי הפעילה (Easy / Medium / Hard)

player // זהות הסוכן הממוחשב (Player1 / Player2)

פלט:

bestMove // אינדקס הבור הנבחר לביצוע המהלך

// שלב 1: בדיקת רשת ביטחון טרמינלית – פותר הסיום (בלעדי ל-Hard)
אם (difficulty == Difficulty.Hard וגם $\text{TotalStonesInPits}(\text{engine}) < 12$) אז:

מהלך פותר $\leftarrow 1$

אם ($\text{true} == \text{EndgameSolver.TrySolve}(\text{engine}, \text{player}, \text{out})$) אז:

החזר מהלך פותר // נמצא פתרון מתמטי מוחלט, מדלגים על ה-Beam

// שלב 2: טעינת פרמטרי החיפוש הדטרמיניסטיים לפי רמת הקושי (עומקי 2/3/4 plies)
 $(\text{horizon}, \text{totalBudget}, K) \leftarrow \text{GetSearchParameters}(\text{difficulty})$

// שלב 3: חלוקת תקציב שוויונית בין מהלכי השורש ובניית תת-העצים
מהלכי שורש $\leftarrow \text{engine.GetValidMoves}()$
 $\text{budget} \leftarrow \text{totalBudget} / \text{Length}$ פנימי לכל מהלך (מהלכי שורש)
 $\text{rootChildren} \leftarrow$ אתחל רשימה ריקה

לכל move בתוך מהלכי שורש תעשה:

$\text{childEngine} \leftarrow \text{engine.Clone}()$
 $\text{childEngine.ApplyMove}(\text{move})$
 $(\text{phase} \leftarrow \text{PhaseAutomaton.Detect}(\text{childEngine}, \text{player}))$
 $(h \leftarrow \text{Heuristic.Evaluate}(\text{childEngine}, \text{player}, \text{phase}))$
 $(\text{node} \leftarrow \text{SearchNode}(\text{childEngine}, \text{rootMove: move}, \text{depth: 1}, h, \text{phase}))$
 $\text{RunBeam}(\text{node}, \text{budget}, K, \text{horizon}, \text{player}, \text{מהלך})$ פנימי לכל מהלך
 $(\text{node.Value} \leftarrow \text{Backup}(\text{node}, \text{player}))$
הוסף את node אל רשימת rootChildren

// שלב 4: הפעלת שגרת דירוג ובחירת הצומת המוביל בעל ערך ה-Backup המרבי
 $(\text{bestNode} \leftarrow \text{Argmax}(\text{rootChildren}, \text{key: node} \rightarrow \text{node.Value}))$
 $\text{chosenMove} \leftarrow \text{bestNode.RootMove}$

// שלב 5: הפעלת שכבת סינון קשיחה – שומר ההגנה (בלעדי ל-Hard)
אם (difficulty == Difficulty.Hard) אז:

$(\text{chosenMove} \leftarrow \text{ApplyCaptureGuard}(\text{engine}, \text{rootChildren}, \text{player}, \text{chosenMove}))$

החזר chosenMove

סיום פונקציה

עדכון יעילות החיפוש:

אלגוריתם החיפוש חסום ומבוקר על ידי שלושה גבולות עליונים מקבילים: אופק עומק מקסימלי המותאם לשלוש רמות הקושי החדשות (2, 3, או 4 plies), תקציב צמתים כולל קשיח לסריקה, וחיתוך רוחב הקרן המורחב (K).

בניגוד לגרסאות הפיתוח המוקדמות, מנוע החיפוש מנותק לחלוטין מרכיבי אקראיות או משעון המערכת הפיזי (DateTime), והרצה שלו מהווה פונקציה דטרמיניסטית טהורה של מצב הלוח הנוכחי המחושב בצורה עקבית ויציבה ללא תלות בעומס המעבד. תכונה זו מחויבת על פי דרישות סעיף 7 בנוהל.

17.4 מחלקת פונקציית ההערכה (Heuristic)

מחלקה סטטית המחשבת את הציון ההיוריסטי עבור מצב לוח נתון מנקודת מבטו של הסוכן. הציון מורכב מגרעין ליבה מרכזי של הפרשי אבנים ומאפיינים אסטרטגיים משתנים המשוקללים דינמית בהתאם לשלב המשחק הנוכחי.

פונקציה	פרמטרים	מחזירה	תפקיד ופעולה	סיבוכיות
Evaluate	engine, perspective, phase	double	מעריכה את איכות מצב הלוח הנוכחי; ערך גבוה מעיד על עמדה יציבה ובטוחה עבור הסוכן הממוחשב.	$O(1)$ (חסומה ב-36 פעולות חישוב קבועות)

פסאודו-קוד עבור המתודה Evaluate:

קלט:

engine // מנוע המשחק ומצב הלוח הנוכחי
perspective // השחקן שמנקודת מבטו מבוצע הניתוח היוריסטי
phase // שלב המשחק כפי שסווג על ידי האוטומט (Opening / Midgame / Endgame / וכו')

פלט:

score // ציון מספרי ממשי המשקף את חוזק העמדה

לוגיקה מבצעת:

אם (engine.IsGameOver == true) אז:

(engine.FinalizeBoard)

החזר (engine.GetScore(perspective) - engine.GetScore(perspective)) * (perspective))
10000

(w1, w2, w3) ← טען וקטור משקולות מותאם לפי phase

הפרש ניקוד ← engine.Store(perspective) - engine.Store(perspective) שחקן (perspective))

איומים אקטיביים ← סרוק בורות ריקים וחשב איומי חטיפה מידיים שלי ושל היריב

איומים פסיביים ← סרוק מהלכי הכנה פוטנציאליים שלי ושל היריב

פוטנציאל חטיפות נטו ← (איומים אקטיביים * 2.0) + (איומים פסיביים * 0.5)

שליטה בלוח ← סך האבנים בורות הרגילים של perspective

בזנוס שרשרת ← 0

אם (phase == GamePhase.CaptureChain) אז:

בזנוס שרשרת ← מספר איומי החטיפה המשולבים * 1.5

קנס רעבון ← 0

אם (סך האבנים בורות היריב ≥ 3) אז:

קנס רעבון ← סך האבנים בורות היריב * 3.0

(w1 ← score * הפרש ניקוד) + (w2 * פוטנציאל חטיפות נטו) + (w3 * שליטה בלוח) + בזנוס

שרשרת - קנס רעבון

החזר score

סיום פונקציה

17.5 מחלקת אוטומט שלבי המשחק (PhaseAutomaton)

מחלקה סטטית המממשת אוטומט מצבים סופי (FSA). היא אחראית לסווג כל מצב לוח נתון לאחד מחמשת שלבי האסטרטגיה של המשחק. הסיווג מבוצע לפי סדר עדיפויות מוגדר, מן המצב הספציפי והקריטי ביותר ועד למצב הכללי.

פונקציה	פרמטרים	מחזירה	תפקיד ופעולה	סיבוכיות
Detect	engine, perspective	GamePhase	מנתחת את מאפייני פיזור האבנים והאיומים על הלוח	$O(1)$ (סריקה קבועה של 14 תאים)

	ומסווגת את שלב המשחק הפעיל.			
--	-----------------------------	--	--	--

פסאודו-קוד עבור המתודה Detect:

קלט:

engine // מנוע המשחק ומצב הלוח הנסרק
perspective // זהות השחקן המבצע את הבדיקה

פלט:

phase // ערך מתוך ה-enum של GamePhase
לוגיקה מבצעת:
שחקן יריב ← החלף שחקן (perspective)
// בדיקת תנאי 1: שלב הרעבת יריב (Starvation)
אם (סך האבנים בבורות של שחקן יריב ≥ 3) אז:

החזר GamePhase.StarvationMode

// בדיקת תנאי 2: שלב שרשרת חטיפות רגישה (CaptureChain)
אם (מספר איומי החטיפה הפעילים על הלוח עבור שני הצדדים ≤ 2) אז:

החזר GamePhase.CaptureChain

// בדיקת תנאי 3: שלב סיום פיזי (Endgame)
אם (סך כל האבנים ב-12 הבורות הרגילים > 12) אז:

החזר GamePhase.Endgame

// בדיקת תנאי 4: שלב אמצע המשחק (Midgame)
אם (סך כל האבנים ב-12 הבורות הרגילים ממוקם בטווח שבין 12 ל-36) אז:

החזר GamePhase.Midgame

// תנאי ברירת מחדל: שלב פתיחה (Opening)
החזר GamePhase.Opening
סיום פונקציה

17.6 מחלקת תור עדיפויות גנרי (MinHeap)

מבנה נתונים עצמאי המממש ערימת מינימום בינארית (Min-Heap) הממופה על גבי מערך דינמי רציף. מבנה זה משמש כחלון ה"פרונטיר" המנהל את צמתי החיפוש השונים ומציף את האלמנטים המבטיחים ביותר לראש התור.

פונקציה	פרמטרים	מחזירה	תפקיד ופעולה	סיבוכיות
Push	item: TItem, priority: TPriority	void	מוסיפה איבר חדש לערימה ומבצעת תיקון כלפי מעלה (BubbleUp) לשמירת תכונות המבנה.	$O(\log n)$
Pop	—	TItem	שולפת ומחזירה את האיבר בעל העדיפות הנמוכה ביותר, ומבצעת תיקון כלפי מטה (BubbleDown).	$O(\log n)$
Peek	—	TItem	מאפשרת צפייה באיבר המינימלי הנמצא בראש הערימה מבלי להסירו ממנה.	$O(1)$
Count	—	int	מאפיין המציג את כמות הרכיבים השמורים בערימה ברגע נתון.	$O(1)$

אכיפת מתודיקה: שגרות העזר BubbleUp ו-BubbleDown מומשו באמצעות רקורסיית-זנב (Tail-recursion) ולא באמצעות לולאות רגילות המכילות עצירת break או דילוג continue, וזאת על מנת לעמוד באופן מלא וקשיח בקריטריונים המתודיים שנאכפים במחונן הפרויקט.

פסאודו-קוד עבור המתודות Push ו-Pop:

פונקציה (Push (item, priority):
הוסף את הזוג (item, priority) אל סוף הרשימה הפנימית
BubbleUp (אורך הרשימה הפנימית - 1)
סיום פונקציה

פונקציה (BubbleUp (index): // תיקון כלפי מעלה – רקורסיית-זנב

אם ($index > 0$) אז:
 אינדקס אב = ($index - 1$) חלקי 2
 אם (עדיפות[$index$] > עדיפות[אינדקס אב]) אז:
 החלף מקומות ברשימה בין $index$ לאינדקס אב
 BubbleUp (אינדקס אב)
 // אם התנאי אינו מתקיים — הפונקציה מסתיימת מעצמה (תנאי העצירה)
 סיום פונקציה

פונקציה Pop():
 אם (הרשימה הפנימית ריקה) אז:
 זרוק חריגה (InvalidOperationException)

איבר מינימלי = הרשימה[0].Item
 אינדקס אחרון = אורך הרשימה הפנימית - 1
 הרשימה[0] = הרשימה[אינדקס אחרון]
 הסר את האיבר האחרון מהרשימה הפנימית
 אם (הרשימה הפנימית אינה ריקה) אז:
 BubbleDown(0)
 החזר איבר מינימלי
 סיום פונקציה

פונקציה BubbleDown(index): // תיקון כלפי מטה – רקורסיית-זנב
 קטן ביותר = $index$
 אינדקס שמאלי = $2 * \text{אינדקס} + 1$ // הכוונה ב"אינדקס" היא ל- $index$ (בעיה עם ישורים)
 אינדקס ימני = $2 * \text{אינדקס} + 2$ // הכוונה ב"אינדקס" היא ל- $index$ (בעיה עם ישורים)

אם (אינדקס שמאלי > אורך הרשימה וגם עדיפות[אינדקס שמאלי] > עדיפות[קטן ביותר]) אז:
 קטן ביותר = אינדקס שמאלי

אם (אינדקס ימני > אורך הרשימה וגם עדיפות[אינדקס ימני] > עדיפות[קטן ביותר]) אז:
 קטן ביותר = אינדקס ימני

אם (קטן ביותר != $index$) אז:
 החלף מקומות ברשימה בין $index$ לקטן ביותר
 BubbleDown (קטן ביותר)
 // אם קטן ביותר == $index$ – הפונקציה מסתיימת מעצמה (תנאי העצירה)
 סיום פונקציה

הערה מבנית (SearchNode): המחלקה SearchNode (המייצגת צומת בעץ החיפוש) משמשת כמבנה הנתונים שתואר בסעיף 12.3. היא מממשת את הממשק Comparable ומגדירה פונקציית מיון פנימית המשווה צמתים לפי ערך h בסדר יורד. תכונה זו נדרשת ומנוצלת ישירות על ידי מנוע החיפוש לצורך ביצוע חיתוך הקרן (Beam Pruning) המהיר בכל רמת עומק.

17.7 מחלקת לוגיקת התצוגה (GameViewModel)

מחלקה ציבורית המהווה את רכיב ה-ViewModel המרכזי של חלון המשחק הראשי. המחלקה מיישמת את עקרון ההפרדה של תבנית MVVM ומתקשרת בצורה חד-כיוונית ומבודדת מול שכבת ה-GameEngine Model (GreedyAgent). המחלקה אינה מכירה פקדים גרפיים של WPF בצורה ישירה; הקישוריות בינה לבין קובץ התצוגה (XAML) מבוצעת על בסיס מנגנון ה-Data Binding המובנה, תוך מימוש ממשק INotifyPropertyChanged ומחלקת הבסיס ObservableObject. המחלקה אחראית על קליטת קלט השחקן, ניהול הודעות הסטטוס, ואיוש תורות המחשב בצורה אסינכרונית.

טבלת המתודות והרכיבים של מחלקת GameViewModel:

פונקציה / מאפיין	פרמטרים	ערך מוחזר	מתודה מבצעת (תפקיד ופעולה)	סיבוכיות זמן ריצה
GameViewModel (בנאי)	difficulty: Difficulty, computerStart s: bool	—	בנאי המחלקה. מאתחל את מופעי ה-Commands הציבוריים, מקשר את מנוע המשחק והסוכן החכם, ויוזם מחזור משחק חדש בהתאם להגדרות הפותח.	$O(1)$
Pit0–Pit5, Pit7–Pit12, Store1, Store2	—	int	14 מאפיינים ציבוריים לקריאה בלבד, המקשרים את חלקי הלוח הגרפיים למערך השלמים הפנימי של המנוע. שינוי ערך במנוע מפעיל קריאה למתודת OnPropertyChan ged לעדכון הממשק.	$O(1)$

מאפיין מצב דינמי המחזיק את הודעת הסטטוס הנוכחית המיועדת להצגה בפס התחתון של המסך (זהות התור, הודעות חטיפה או סיום).	string	—	—	StatusMessage
פקודה המופעלת בעת לחיצת המשתמש על בור חוקי. מבצעת ולידציה, מפעילה את המהלך במנוע, מעדכנת את חלון התצוגה ומגררת אסינכרונית את תור המחשב.	—	pitIndex: int	—	PlayPitCommand
שגרה אסינכרונית המנהלת את תור הסוכן הממוחשב ברקע (Task.Run) על גבי Thread Pool נפרד. תומכת באסימוני ביטול ורצף תורות נוספים של ה-AI.	Task	—	—	MaybeRunComputerTurnAsync
פקודה המבטלת את אסימון הביטול הנוכחי (במידה והמחשב באמצע חישוב), ומפעילה מנוע משחק חדש תחת אותם מאפייני	—	—	—	NewGameCommand

	קושי ויזום פותח פעילים.			
(O(1	מתודה ציבורית המופעלת על ידי ה-Code-Behind של חלון ה-Wpf בעת חזרה ממסך ה-Setup; מעדכנת את המשתנים הניהוליים ומזניקה סבב חדש.	void	difficulty: Difficulty, computerStarts: bool	Reconfigure
O(1) (14 עדכונים קבועים)	מתודת טיפול באירועי המנוע. מפעילה שידור רענון לכל 14 מאפייני הלוח ומעדכנת את הודעת ה-StatusMessage בהתאם לסוג הפעולה שהתרחשה.	void	—	OnBoardChanged

פסאודו-קוד עבור המתודה StartNewGame:

StartNewGame():

לוגיקה מבצעת:

אם (cancellationTokenSource אינו ריק) אז:

הפעל ביטול (cancellationTokenSource.Cancel)

cancellationTokenSource ← יצירת CancellationToknSource חדש
אם (engine אינו ריק) אז:

הסר רישום מאירוע engine.BoardChanged

שחקן פותח ← (computerStarts == true) ? Player2 : Player1
engine ← יצירת GameEngine חדש (שחקן פותח)
הרשם לאירוע engine.BoardChanged (מפעיל את המתודה OnBoardChanged)
agent ← יצירת GreedyAgent חדש (Player2, difficulty)
אם (engine.CurrentPlayer == Player1) אז:

". StatusMessage ← "Player 1's turn

אחרת:

"... StatusMessage ← "Computer's turn
הפעל אסינכרונית את המתודה MaybeRunComputerTurnAsync()

שדר עדכון עבור כל מאפייני הלוח ומצבי הפקדים (OnPropertyChanged)
סיום פונקציה

פסאודו-קוד עבור המתודה MaybeRunComputerTurnAsync:

MaybeRunComputerTurnAsync():

פלט:

Task // עצם משימה המייצג את פעולת החישוב ברקע
לוגיקה מבצעת:
אם (engine.IsGameOver == true או engine.CurrentPlayer == Player1) אז:

צא מהפונקציה

אסימון ביטול ← cancellationTokenSource.Token
נסה:

IsComputerThinking ← true
רענן מצב פקודות (RaiseCanExecuteChanged) // חסימת כפתורי הממשק ללחיצה

// לולאת ניהול תור המחשב התומכת ברצף תורות נוספים (Chaining) של הסוכן
כל עוד (engine.IsGameOver == false וגם engine.CurrentPlayer == Player2) תעשה:

"...StatusMessage ← "Computer is thinking
המתן אסינכרונית (Task.Delay) למשך 2500 מילישניות לצורך משוב ויזואלי
ודא שאינדקס אסימון ביטול לא הופעל (ThrowIfCancellationRequested)
// הרצת חישוב האלגוריתם הכבד על גבי Thread נפרד מה-Thread Pool
מהלך נבחר ← הפעל ברקע (Task.Run) את (agent.CalculateMove(engine))
ודא שאינדקס אסימון ביטול לא הופעל
engine.ApplyMove (מהלך נבחר) // הפעלת המהלך משנה את הלוח ומפעילה את
OnBoardChanged

תפוס חריגת ביטול (OperationCanceledException):

צא מהפונקציה בצורה שקטה (המשתמש אתחל משחק באמצע חישוב)
לבסוף:

IsComputerThinking ← false

רענן מצב פקודות (RaiseCanExecuteChanged) // שחרור הלוח חזרה למשתמש
סיום פונקציה

17.8 מחלקת לוגיקת מסך הפתיחה (SetupViewModel)

מחלקה ציבורית המנהלת את לוגיקת ממשק המשתמש של חלון הגדרות המשחק (SetupWindow). המחלקה יורשת ממחלקת התשתית ObservableObject, ותפקידה הארכיטקטוני הוא לאסוף ולרכז את בחירות המשתמש בנוגע לרמת הקושי המבוקשת וזהות השחקן שיבצע את המהלך הראשון (יוזם המשחק), ולתרגם אותן בצורה מאוזנת עבור התוכנית הראשית.

טבלת המאפיינים והמתודות של מחלקת SetupViewModel:

פונקציה / מאפיין	פרמטרים	ערך מוחזר	מתודה מבצעת (תפקיד ופעולה)	סיבוכיות זמן ריצה
IsEasy / IsMedium / IsHard	—	bool	שלושה מאפיינים בוליאניים בלעדיים המקושרים בקישור דו-כיווני (TwoWay) לכפתורי הרדיו של רמות הקושי במסך. שינוי אחד מהם מעדכן אוטומטית את מאפיין ה-SelectedDifficulty.	O(1)
PlayerStarts / ComputerStarts	—	bool	צמד מאפיינים בוליאניים בלעדיים המקושרים בקישור דו-כיווני לכפתורי הרדיו של בחירת יוזם המהלך הראשון. משפיעים על הפעלת רענון הממשק של שני הרכיבים.	O(1)

SelectedDifficulty	—	Difficulty	מאפיין ציבורי לקריאה בלבד (Get). מנתח את מצב שלושת המשתנים הבוליאניים של הקושי ומחזיר את ערך ה-enum הנדרש עבור מחלקות ה-Core.	O(1)
--------------------	---	------------	---	------

כפתורי Start ו-Cancel של מסך הפתיחה אינם פקודות ViewModel, אלא מטופלים בשכבת התצוגה: כפתור Start מחובר למטפל אירוע OnStartClick ב-Code-Behind של SetupWindow (המגדיר DialogResult = true). כפתור Cancel מוגדר ב-XAML עם IsCancel="True", המגדיר אוטומטית DialogResult = false.

17.9 מחלקת פותר מצבי הסיום (EndgameSolver)

מחלקה סטטית ופנימית בשם internal static class EndgameSolver, המממשת מנוע חיפוש שני, נפרד ועצמאי לחלוטין ממנוע הקרן המקורב. בעוד שמנוע הקרן עוצר באופק עומק מוגדר ומעריך את העמדה באמצעות נוסחה היוריסטית, פותר הסיום נכנס לפעולה בשלב הסיום הרשמי של המשחק (כאשר סך כל האבנים ב-12 הבורות הרגילים יורד ל-12 ומטה אבנים) ומבצע סריקה ממצה ומלאה של עץ המשחק עד להגעה לעלי המשחק הסופיים המדויקים.

טבלת המתודות של מחלקת EndgameSolver:

פונקציה	פרמטרים	ערך מוחזר	מתודה מבצעת (תפקיד ופעולה)	סיבוכיות זמן ריצה
TrySolve	engine: GameEngine, player: Player, out bestMove: int	bool	נקודת הכניסה הציבורית של פותר הסיום. מאתחלת את טבלת ה-Memoization, ומריצה מעבר רקורסיבי על כל מהלכי השורש החוקיים. מחזירה true והמהלך המנצח במידה והפיצוח הושלם בתקציב.	O(S) כאשר S הוא מספר המצבים וחסום ב-250,000

המתודה הרקורסית הפנימית המבצעת את פריסת עץ המשחק לאחר. מפעילה שכבת מקסימום לסוכן וחיזוי חמדני יחיד ליריב, ושומרת את הערכים המוחלטים בטבלה.	int	engine: GameEngine, player: Player, isAgentTurn: bool, memo: Dictionary<string, int>, ref nodeCount: int	Solve
פונקציית עזר פנימית המקודדת ומחברת את 14 ערכי התאים בלוח וזהות התור לכדי מחרוזת טקסט רציפה המופרדת בפסיקים, לשימוש כמפתח בטבלה.	string	engine: GameEngine, isAgentTurn: bool	BuildKey

פסאודו-קוד עבור המתודה TrySolve:

:(TrySolve (engine, player, out bestMove

קלט:

engine // מנוע המשחק ומצב הלוח המקורי לפני ביצוע המהלך
player // זהות השחקן הנוכחי (הסוכן החכם)

פלט:

bestMove // אינדקס המהלך המנצח המדויק (מוחזר כפרמטר פלט)
ערך מוחזר בוליאני // true אם נמצא פתרון מתמטי ודאי, false אחרת

bestMove ← -1

סך אבנים ← סך כל האבנים ב-12 הבורות הרגילים בלוח

// הפותר כשיר לפעולה רק בשלב הסיום הרשמי של המשחק
אם (סך אבנים < 12 או engine.IsGameOver == true) אז:

החזר false

memoTable ← אתחל מילון גיבוב ריק מסוג <Dictionary<string, int>

nodeCount ← 0
מהלכים חוקיים ← engine.GetValidMoves()
ערך מיטבי ← מינוס אינסוף

לכל מהלך בתוך מהלכים חוקיים תעשה:

עותק מדמה ← engine.Clone()
עותק מדמה.ApplyMove(מהלך)
IsAgentTurn ← (CurrentPlayer == player)
ערך מחושב ← SolveRecursive(עותק מדמה, player, IsAgentTurn, memoTable, nodeCount)

// הגנה מפני תקיעה — אם חרגנו מתקציב הבטיחות, הפותר קורס ומבצע נסיגה ל-Beam
אם (nodeCount > 250000) אז:

החזר false

אם (ערך מחושב < ערך מיטבי) אז:

ערך מיטבי ← ערך מחושב
bestMove ← מהלך

החזר true
סיום פונקציה

ארכיטקטורה חד צדדית מנומקת:

למרות שפותר הסיום סורק את מרחב המשחק עד להגעה לעלים הסופיים המדויקים, הוא אינו עושה שימוש בשכבת מינימיזציה (min) מלאה ואינו מפעיל פריסה רוחבית בתורו של היריב. במקום זאת, הוא שומר על אחדות עם מנוע ה-Beam ומסמלצת את צעדי הנגד כמהלך בודד חמדני ומדויק מתוך המתודה המשותפת SimulateGreedyOpponentMove. תכונה מבנית זו מוציאה את הפותר לחלוטין מגדר אלגוריתם המינימקס האסור על פי סעיף 13 בנוהל.

ההנחה ההנדסית העומדת בבסיס מודל זה היא כי הסוכן משחק מול שחקן אנושי מציאותי הפועל מתוך חמדנות מקומית קצרת ראי, ולכן חובה לחזות תגובה המבוססת על המודל החמדני הקיים ולא תגובה אופטימלית מתמטית מנותקת. במידה והפותר מצליח להשלים את הסריקה בתוך גבולות תקציב הבטיחות, המהלך מבוצע ישירות ומנוע ה-Beam וה-Capture Guard מנוטרלים ומדולגים כליל, שכן אין צורך בקירובים או ברשתות ביטחון היריסטיות כאשר קיים פתרון טרמינלי מושלם.

18. התוכנית הראשית

התוכנית הראשית של הפרויקט - הפונקציה המנהלת את מחזור החיים של היישום כולו, היא המתודה `App.OnStartup`, המהווה את נקודת הכניסה הרשמית של אפליקציית ה-WPF. היא אחראית על ניהול זרימת ההפעלה: הצגת מסך הגדרות הקושי, ולאחר קבלת בחירת המשתמש, בנייה והצגה של חלון המשחק הראשי. להלן קוד-הפסאודו המתאר את פעולתה:

פונקציה `OnStartup()`:

לוגיקה מבצעת:

```
Application.Current.ShutdownMode = ShutdownMode.OnExplicitShutdown
// מונע סגירת האפליקציה בשבריר השנייה שבין סגירת מסך ההגדרות לפתיחת חלון המשחק
```

```
setupVM ← יצירת SetupViewModel חדש ואיושו כ-DataContext של החלון
setupWindow ← יצירת חלון הגדרות חדש מסוג SetupWindow
אם (setupWindow.ShowDialog() == true) אז:
```

```
קושי נבחר ← setupVM.SelectedDifficulty
המחשב מתחיל ← setupVM.ComputerStarts
main ← יצירת חלון משחק ראשי מסוג MainWindow (קושי נבחר, המחשב מתחיל)
main ← Application.Current.MainWindow
main.Show()
Application.Current.ShutdownMode ← ShutdownMode.OnLastWindowClose
```

אחרת:

```
Application.Current.ShutdownMode ← ShutdownMode.OnExplicitShutdown
// המשתמש ביטל את חלון הפתיחה, סגירת היישום
סיום פונקציה
```

מניעת מצבי מירוץ (Race Conditions):

מנגנון ה-`ShutdownMode` מטופל בקוד באופן ידני ומפורש. ברירת המחדל של WPF היא לסגור את האפליקציה ברגע שחלון הפתיחה נסגר. מאחר שבין רגע סגירתו של מסך ה-`Setup` לבין רגע פתיחתו של ה-`MainWindow` הראשי ישנו שבריר שנייה שבו אין אף חלון פתוח במערכת, WPF עלולה לפרש זאת בטעות כסיום עבודה ולסגור את התוכנה כולה. העברת המצב ל-`OnExplicitShutdown` מונעת זאת, והחזרתו ל-`OnLastWindowClose` לאחר פתיחת החלון הראשי מבטיחה שהאפליקציה תיסגר בצורה נקייה ומסודרת ברגע שהמשתמש יסגור את חלון המשחק.

19. מדריך למשתמש

סעיף זה מתאר, שלב אחר שלב, כיצד להפעיל את תוכנת המשחק וכיצד לתפעל אותה מנקודת מבטו של המשתמש הקצה.

19.1 הפעלת התוכנה

התוכנה מיושמת כאפליקציית WPF שולחנית (Windows). עם הפעלת קובץ ההרצה, נפתח תחילה מסך הפתיחה (חלון ההגדרות). רק לאחר אישור ההגדרות וכללי המשחק, יופעל חלון המשחק הראשי. התוכנה אינה דורשת התקנה מיוחדת, חיבור לרשת או הרשאות מנהל.

19.2 מסך הפתיחה והגדרת קבוצת המשחק (SetupWindow)

במסך הפתיחה, על המשתמש לקבוע שני מאפיינים מרכזיים לפני תחילת סבב המשחק:

- **רמת קושי (Difficulty):** בחירה מבין שלוש רמות קושי שונות המשפיעות על אופק החיפוש של הסוכן הממוחשב:

- **Easy:** המחשב מנתח לעומק של **2 plies** קדימה בלבד - רמה בסיסית המתאימה להתרשמות ראשונית.
- **Medium:** המחשב מנתח לעומק של **3 plies** קדימה.
- **Hard:** הרמה המאתגרת ביותר. המחשב מנתח לעומק של **4 plies** קדימה ובנוסף מפעיל מנגנון שומר-הגנה קשיח (Capture Guard) למניעת מהלכים המפקירים אבנים.

- **זהות השחקן המתחיל (Who plays first):** בורר דו-מצבי לקביעת השחקן שיבצע את המהלך הראשון:

- **I play first:** (ברירת המחדל) השחקן האנושי מבצע את המהלך הראשון.
- **Computer plays first:** הסוכן הממוחשב יבצע את מהלך הפתיחה באופן אוטומטי.

לחיצה על כפתור **Start** תסגור את מסך הפתיחה ותפתח את חלון המשחק בהתאם לבחירות שבוצעו. סגירת החלון או לחיצה על **Cancel** תביא לסיום ריצת התוכנה.

19.3 חלון המשחק הראשי (MainWindow)

חלון המשחק מציג את לוח המנקה, המורכב מ-14 תאים בסך הכל:

- **השורה התחתונה:** שייכת לשחקן האנושי (שחקן 1), והבורות בה ממוספרים מ-1 עד 6 (מימין לשמאל).
- **השורה העליונה:** שייכת לסוכן הממוחשב (שחקן 2).
- **המאגרים (קופות):** המאגר הימני משמש כקופה של השחקן האנושי, והמאגר השמאלי משמש כקופה של המחשב. מספר האבנים בכל תא מוצג באופן ספרתי על גבי הרכיב הגרפי, והתצוגה מתעדכנת אוטומטית בסיום כל מהלך. השורה של השחקן שזהו תורו כעת תודגש ויזואלית על גבי המסך.

19.4 ביצוע תור (מהלך)

- **בתור השחקן האנושי:** על השחקן ללחוץ באמצעות עכבר המחשב על אחד הבורות הלא-ריקים בשורה שלו. המערכת תבצע אוטומטית את פעולת הזריעה (פיזור האבנים נגד כיוון השעון) ותחשב חטיפות או תורות נוספים בהתאם לחוקים. בורות ריקים או בורות בצד של המחשב חסומים ללחיצה, והמערכת תמנע מהלכים לא חוקיים.
- **בתור המחשב:** עם העברת התור למחשב, תופיע הודעה בשורת הסטטוס: "Computer... is thinking". המחשב ישהה את פעולתו למשך זמן קצר (כדי לאפשר מעקב ויזואלי אחר המשחק), יבצע את המהלך הנבחר, והלוח יתעדכן. בזמן חשיבת המחשב, לחצני הלוח מושבתים זמנית למניעת הפרעות.

19.5 הודעות סטטוס

בתחתית חלון המשחק קיים פס סטטוס המיידע את המשתמש בכל רגע נתון לגבי מצב המשחק: של מי התור הנוכחי, האם בוצעה חטיפת אבנים וכמה אבנים נחטפו, והאם הוענק תור נוסף לאחד הצדדים. הודעות הסטטוס מושהות קלות לפני החלפתן כדי לאפשר למשתמש לקרוא אותן בנוחות.

19.6 כפתורי השליטה והניווט

בכותרת חלון המשחק קיימים שני כפתורים קבועים:

- **New Game:** מאתחל את לוח המשחק ומחזיר את האבנים למצב ההתחלתי (4 אבנים בכל בור) תחת אותן הגדרות קושי ויזום שנקבעו מראש.
- **Setup:** מאפשר לחזור למסך הפתיחה הראשי מבלי לסגור את התוכנה כולה. לחיצה על כפתור זה תסגור את חלון המשחק הנוכחי ותציג מחדש את חלון ההגדרות. אם המשתמש יתחרט וילחץ על **Cancel** בחלון ההגדרות שנפתח, הוא יוחזר למשחק הפעיל שלו בדיוק באותו המצב שבו עזב אותו.

19.7 סיום המשחק

המשחק מגיע לסיומו הרשמי כאשר ששת הבורות באחד הצדדים מתרוקנים לחלוטין. ברגע זה, כל האבנים שנותרו בבורות של הצד השני נאספות אוטומטית אל המאגר של בעל הצד. המערכת תציג תיבת הודעה (MessageBox) המכריזה על התוצאה הסופית: ניצחון לשחקן, ניצחון למחשב או תיקו, יחד עם פירוט הנקודות. לאחר סגירת ההודעה, ניתן לבחור להתחיל משחק חדש (New Game) או לשנות הגדרות (Setup).

19.8 הרצה דרך ממשק שורת הפקודה (Console - אופציונלי)

בנוסף לממשק הגרפי, קיימת בפרויקט מחלקת הרצה חלופית (MancalaProject.Console). ממשק זה מיועד בעיקר לצורך הרצת בדיקות רגרסיה וכיול של מנוע ה-AI, ואינו מיועד למשתמשי קצה. חלון הקונסול מציג את הלוח בצורה טקסטואלית, הבחירה ברמת הקושי מתבצעת על ידי הקשת המספרים 1-3, וביצוע המהלך נעשה על ידי הקשת מספר הבור הרצוי (1-6).